

Game Programming using C++

**SDC GPWC,
Faculty of Engineering and Technology (ITER)
SOA (Deemed to be) University, BBSR, Odisha**

2025-2026

Name of the Subject	Game Programming with C++
Subject Code	CSE 3545
Grading Pattern	1
Subject Credits	4
Course	CSE, 6th Semester

Introduction to course

To equip students with the foundational and advanced concepts of game development using C++, including the application of object-oriented programming, game physics, graphics rendering, artificial intelligence, and gameplay mechanics. The course aims to develop students' problem-solving and creative skills by designing and implementing interactive and immersive gaming experiences. Students will also learn to use industry-standard tools and libraries to create efficient and scalable game applications.

Program Outcomes (POs)

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. **Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. **Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.
6. **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. **Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. **Individual and teamwork:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

11. **Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12. **Life-long learning:** Recognize the need for and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Program Specific Outcomes (PSOs)

1. **PSO 1:** The ability to understand, analyze and develop computer programs in the areas related to business intelligence, web design and networking for efficient design of computer-based systems of varying complexities.
2. **PSO 2:** The ability to apply standard practices and strategies in software development using open-ended programming environments to deliver a quality product for business success.

Course Outcome (COs)

Students will be able to

1. **CO1: write, compile and execute** simple C++ programs to solve computational problems.
2. **CO2: become familiar** with basic games using SFML and Visual Studio, implementing sprites, animations, player inputs, and Heads-Up Display (HUD) elements.
3. **CO3: implementing** Game Mechanics and Physics like collision detection, pickups, bullets, and sound effects while applying dynamic collision detection and basic physics concepts.
4. **CO4: develop** advanced game features, such as zombie shooter games, using sprite sheets, vertex arrays, texture management, and layering views.
5. **CO5: apply** advanced object-oriented programming concepts like inheritance, polymorphism, and abstraction to improve code management and game scalability.
6. **CO6: implement** file I/O, sound effects, and refining the game's HUD and UI, while optimizing performance using C++ references, pointers, and the Standard Template Library (STL).

Course Structure

Grading Pattern

Type	Category		Mark wait	
	Attendance (if >75):		05	
INTERNAL	Assignments and Quizzes :		20	
	Mid-term:		15 ----->	Mapped from 30
-----	-----	-----	-----	-----
	End-term:	LAB test	15	
EXTERNAL		END-sem	45 ----->	Mapped from 60
		Total	100	

Credits and Course Format

- Grading Pattern: 1
- Credits: 4
- 3 Classes/week, 1hr/class
- 1 Labs/week, 2hr/Lab

Course Curriculum

CSE 3545	Game Programming with C++	4	1
C++, SFML, Visual Studio, and Starting the First Game, Variables, Operators, and Decisions – Animating Sprites, C++ Strings and SFML Time – Player Input and HUD, Loops, Array, Switches, Collisions, Sound, and End Conditions – Making the Game Playable, Object-Oriented Programming – Starting the Pong Game, Dynamic Collision Detection and Physics – Finishing the Pong Game, SFML Views – Starting the Zombie Shooter Game, C++ References, Sprite Sheets, and Vertex Arrays, Pointers, the Standard Template Library, and Texture Management, Collision Detection, Pickups, and Bullets, Layering Views and Implementing the HUD, Sound Effects, File I/O, and Finishing the Game, Abstraction and Code Management – Making Better Use of OOP, Advanced OOP – Inheritance and Polymorphism.		Textbook – Beginning C++ Game Programming by Horton, Packt Publishing	
		Weekly Course Format: 3L - 2P	

Books for reference

1. **J. Horton, Beginning C++ game programming: learn to program with C++ by building fun games, Second edition. Place of publication not identified: Packt Publishing, 2019.**
2. **E. Balagurusamy, Object oriented programming with C++, Eighth Edition. New Delhi: McGraw Hill Education (India) Private Limited, 2021.**

Introduction to Evaluation strategy

Mid-semester Question Format :

- 5 questions each carrying 3 bits
- each bit will be of 2 marks
- Total of 30 marks
- Exam will be conducted for 2 Hrs

External Lab Question Format

- 3 Question Programming Based
- Each carrying 5 Marks
- Total of 15 marks.
- Exam will be conducted for 2 Hrs

End-Semester Question Format

- 10 questions each carrying 3 bits
- each bit will be of 2 marks
- Total of 60 marks
- Exam will be conducted for 3 Hrs

Quiz Test (Minimum 4)

- 30 questions each carrying 1 mark
- Total Mark 30
- Exam will be conducted for 30 Mins
- Will be done in online mode. (Present or absent have to attend the quiz test.)
- Average of all quizzes will be considered for final evaluation.

Assignment Submission

- There will be Minimum 10 assignments which students have to submit each carrying 10 marks.
- The students have to complete the assignment and upload the scanned copy of it in shared google form along with the screenshot of the output.
- At the end of the semester before the External Lab exam the students have to submit the complete assignments in physical format.
- Assignments are to be submitted within the given deadline failing to which 4 marks will be deducted.
- Students may be asked to redo in case any unfair means is observed.

Introduction to C++

Introduction to C++

C++ is a general-purpose, high-performance programming language that supports object-oriented, procedural, and generic programming paradigms. It was developed by **Bjarne Stroustrup** in 1979 as an extension of the C programming language, adding features such as classes and object-oriented programming.

Key Features of C++:

1. **Object-Oriented Programming:** Supports concepts like classes, objects, inheritance, polymorphism, encapsulation, and abstraction.
2. **Portability:** Code written in C++ can run on different platforms with minimal or no modifications.
3. **Efficiency and Performance:** Provides direct access to hardware and system-level resources, making it ideal for performance-critical applications.
4. **Standard Template Library (STL):** A rich library of templates for data structures (e.g., vectors, lists), algorithms (e.g., sorting, searching), and iterators.
5. **Low-Level Manipulation:** Allows direct memory manipulation using pointers, which is helpful in system programming.
6. **Extensibility:** Users can define their own types and overload operators for custom behavior.
7. **Rich Library Support:** Includes built-in libraries for file handling, data manipulation, and more.

What is the difference between C and C++?

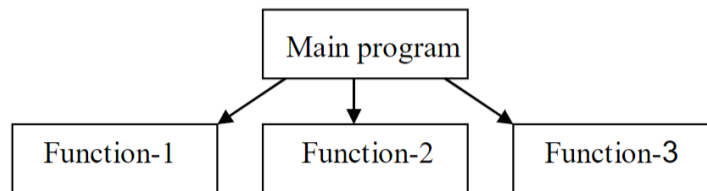
- C is a **procedural programming language**, while C++ is a **multi-paradigm language** that supports both procedural and object-oriented programming.

Why C++ Was Developed?

- C was a popular and efficient programming language for system-level programming, but it lacked features that made it suitable for solving complex, large-scale problems.
- At the same time **Simula** programming language was developed in the 1960s by **Ole-Johan Dahl** and **Kristen Nygaard** at the Norwegian Computing Center in Oslo, Norway. It is widely regarded as the first **object-oriented programming (OOP)** language. Initially designed for simulation applications, Simula allowed users to model real-world systems, such as queuing systems or process flows.
- Bjarne Stroustrup, the creator of C++, acknowledged that Simula's class concept was a key inspiration.

Procedure Oriented Programming vs Object Oriented Programming

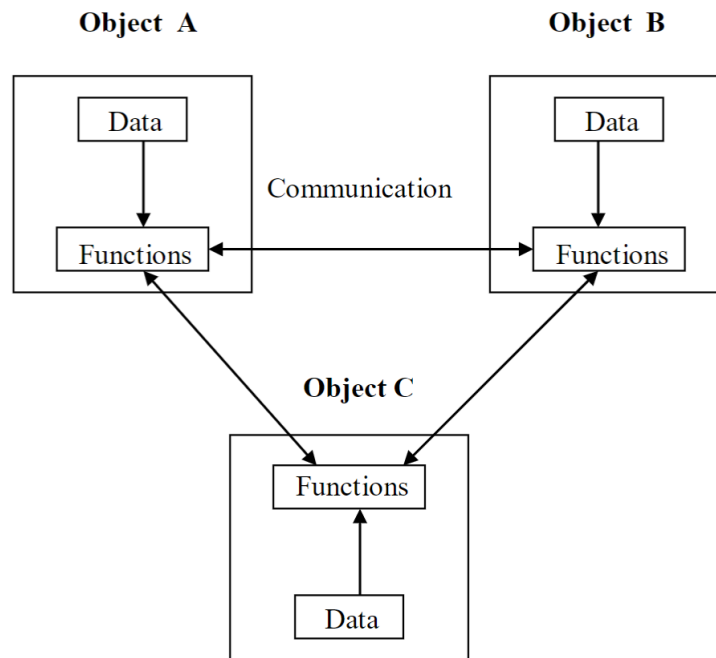
Procedure Oriented Programming (POP) focuses on functions or procedures to achieve tasks. Programs in POP follow a linear and sequential approach, with data being shared globally among functions. This can lead to challenges in maintaining and securing data. Modularity is achieved by dividing the program into smaller, independent functions. However, code reusability is limited, making scalability difficult in larger projects. POP is typically used for simpler applications, with examples of POP languages including C and Fortran.



The disadvantage of the procedure oriented programming languages is:

1. Global data access
2. It does not model real word problem very well
3. No data hiding

On the other hand, **Object Oriented Programming (OOP)** emphasizes objects, which bundle data and behavior together. Key concepts in OOP include classes, objects, inheritance, encapsulation, polymorphism, and abstraction. This approach allows better data security by encapsulating data within objects and restricting access. OOP ensures modularity through classes and objects, improving code reusability and scalability. It closely maps to real-world entities, making it intuitive for designing complex applications. OOP languages like Java, Python, and C++ are ideal for developing enterprise-level software, games, and web applications.



Features of the Object Oriented programming

1. Emphasis is on doing rather than procedure.
2. programs are divided into what are known as objects.
3. Data structures are designed such that they characterize the objects.
4. Functions that operate on the data of an object are tied together in the data structure.
5. Data is hidden and can't be accessed by external functions.
6. Objects may communicate with each other through functions.
7. New data and functions can be easily added.
8. Follows bottom-up approach in program design.

Feature	Procedure-Oriented Programming (POP)	Object-Oriented Programming (OOP)
Focus	Functions/Procedures	Objects
Key Concept	Functions	Classes, Objects
Data Security	Less secure	Encapsulation for security
Modularity	Function-based	Class and Object-based
Reusability	Low	High
Scalability	Difficult	Easier
Real-world Mapping	No	Yes
Example Languages	C, Fortran	Java, Python, C++

Class vs Structure

A **structure** is a user-defined data type that groups related variables (fields) together under one name. It is a value type, typically used for creating lightweight objects that do not require inheritance or complex behavior. Structures are designed to store and manipulate simple data efficiently. They are often used in procedural and object-oriented programming for small, self-contained data entities.

A **class** is a blueprint for creating objects in object-oriented programming. It defines a reference type that encapsulates data (fields) and behavior (methods) into a single entity. Classes are designed for creating complex, reusable, and scalable objects that can interact with each other. They support advanced object-oriented features like inheritance, polymorphism, and encapsulation.

Feature	Class	Structure
Type	Reference	Value
Memory Allocation	Heap	Stack
Inheritance	Supported	Not supported
Default Access	Private	Public
Performance	Slower (heap management)	Faster (stack-based)
Use Case	Complex objects, large data	Lightweight objects, small data
Constructor	Parameterless by default	Explicit, all fields initialized

Some Basic Concepts of OOP

1. Classes

- A **class** is a blueprint for creating objects
- A class is a new data type that contains member variables and member functions that operate on the variables.

2. Objects

- Objects are the basic run-time entities.
- They are the instance of a class.

3. Data abstraction

- Abstraction refers to the act of representing essential features without including the back ground details or explanations.

4. Data Encapsulation

- The wrapping up of data and function into a single unit (called class) is known as encapsulation. The data is not accessible to the outside world and only those functions which are wrapped in the class can access it.

5. Inheritance

- Inheritance is the process by which objects of one class acquire the properties of another class. In the concept of inheritance provides the idea of reusability. This mean that we can add additional features to an existing class with out modifying it.

6. Polymorphism

- Polymorphism means the ability to take more than one form. An operation may exhibit different instance. The behavior depends upon the type of data used in the operation.

7. Dynamic binding

- Binding refers to the linking of a procedure call to the code to be executed in response to the call. Dynamic binding means the code associated with a given procedure call is not known until the time of the call at run-time.

8. Message passing

- OOP consists of a set of objects that communicate with each other.
- A message for an object is a request for execution of a procedure and therefore will invoke a function (procedure) in the receiving object that generates the desired result.

First C++ Program

```
#include <iostream> // Required for input/output operations
int main() {
    std::cout << "Hello, World!" << std::endl; // Print message to the
    console
    return 0; // Indicate that the program ended successfully
}
```

Explanation:

1. `#include <iostream>` : This is a preprocessor directive to include the Input/Output stream library.
2. `int main()` : The starting point of the program. Every C++ program must have a `main()` function.
3. The `std` **namespace** exists in the **C++ Standard Library**, which is part of the compiler's implementation of the C++ language.
4. The `::` operator in C++ is called the **scope resolution operator**. It is used to specify the scope of a particular identifier, such as a variable, function, or class, when there are multiple possible scopes.
5. `cout` is an instance of the `ostream` class defined in `std` namespace.
6. `std::cout` : Used to print output to the console.
7. The `<<` operator in C++ is called the **stream insertion operator** when used with **output streams**, such as `std::cout` . It is used to send data to the output stream for display on the console.
8. `"Hello, World!"` : The message that will be displayed on the console.
9. `endl` is a function. However, it behaves like an object when used with `std::cout` because it acts as an argument to the `operator<<` function. Specifically, `operator<<` is **overloaded** to accept manipulators (like `std::endl`).
10. `std::endl` : Ends the current line and flushes the output buffer.
11. `return 0;` : Indicates that the program executed successfully.

How to Run:

1. Save the code in a file with a `.cpp` extension (e.g., `hello_world.cpp`).
2. Compile the program using a C++ compiler

```
g++ hello_world.cpp -o hello_world
```

3. Run the compiled program

```
./hello_world
```

Output

```
Hello, World!
```

What is a namespace ? Why it is Necessary ?

A **namespace** in C++ is a way to group logically related classes, functions, variables, and other identifiers under a unique name. It is used to organize code and avoid naming conflicts in large programs or when using multiple libraries.

Namespaces were introduced in C++ to solve the problem of **name collisions** in large projects. Without namespaces, functions, variables, or classes with the same name in different parts of the program or libraries would cause ambiguity and errors.

Advantages of Namespaces

1. **Prevents Name Collisions:** Avoids ambiguity when different parts of the program or libraries define entities with the same name.
2. **Improves Code Readability:** Groups related entities logically under a meaningful name.
3. **Enhances Modularity:** Encourages splitting large codebases into smaller, independent modules.
4. **Facilitates Library Design:** Makes it easier to integrate multiple libraries into the same project.

Removing the `std` prefix for easy coding

We can directly use Standard Library components without the `std::` prefix, if we give directive to compiler that we are going to use `std` namespace and the risk of name conflicts is on the programmer. The directive used is

```
using namespace std;
```

Hence, the above code will be modified to

```
#include <iostream>
using namespace std;
int main() {
    cout << "Hello, World!" << endl;
```

```
    return 0;  
}
```

Standard Input and Output Stream

In **C++**, `cin` and `cout` are the standard input and output streams, respectively, provided by the **Standard Input/Output Library** (`<iostream>`). They are used to perform basic input and output operations in C++ programs.

`cout` (Console Output):

`cout` stands for "**character output**", used to print data to the console. It uses the **standard output stream** (`stdout`) for display.

- **Purpose:** Used to display/output data to the console (standard output).
- **Syntax:** `cout << expression;`
- The `<<` **operator**, called the "insertion operator," sends the data to the output stream.

Example

```
#include <iostream> // Required for cout and cin
using namespace std;

int main() {
    cout << "Hello, World!" << endl; // Output message
    cout << "The value of 5 + 3 is: " << 5 + 3 << endl; // Output
    expression
    return 0;
}
```

Output

```
Hello, World!
The value of 5 + 3 is: 8
```

Chaining Output

You can chain multiple outputs using the `<<` operator. This makes the code concise and expressive.

```
cout << "Name: " << "John Doe" << ", Age: " << 25 << endl;
```

Output

Name: John Doe, Age: 25

Formatting Output

To improve readability, you can format the output using **manipulators** from the `<iomanip>` library:

- `setw(n)` : Sets the width of the output field.
- `fixed` : Displays floating-point numbers in fixed-point notation.
- `setprecision(n)` : Sets the number of digits after the decimal point.
- `left` / `right` : Aligns the output to the left or right.

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    double price = 123.456;
    cout << fixed << setprecision(2); // Fixed-point with 2 decimals
    cout << "Price: $" << price << endl;

    cout << setw(10) << left << "Item" << setw(8) << right << "Price" <<
endl;
    cout << setw(10) << left << "Book" << setw(8) << right << "$12.99" <<
endl;

    return 0;
}
```

Output

```
Price: $123.46
Item      Price
Book      $12.99
```

Escape Sequences

Special characters can be displayed using **escape sequences**:

Escape Sequences in C++ (cout)

Escape Sequence	Meaning	Example Code	Output
\n	Newline	cout << "Hello\nWorld!";	Hello World!
\t	Horizontal tab	cout << "Hello\tWorld!";	Hello World!
\\	Backslash	cout << "This is a backslash: \\";	This is a backslash:
\"	Double quote	cout << "He said, \"Hi!\\"";	He said, "Hi!"
\'	Single quote	cout << "It\'s a sunny day.";	It's a sunny day.
\a	Alert (bell sound, may not work on all systems)	cout << "\a";	(Bell sound)
\b	Backspace (moves cursor back one position)	cout << "Hello\bWorld!";	HellWorld!
\r	Carriage return (moves cursor to start of line)	cout << "Hello\rWorld!";	World!
\f	Form feed (moves cursor to next page, rarely used)	cout << "Hello\fWorld!";	(May vary by system)
\v	Vertical tab	cout << "Hello\vWorld!";	Hello World!
\?	Question mark	cout << "Why is this here: \?";	Why is this here: ?
\0	Null character (end of a string)	cout << "This is\0null";	This is

Example: Demonstrating Escape Sequences

```
#include <iostream>
using namespace std;

int main() {
    cout << "Newline: Hello\nWorld!\n";
    cout << "Tab: Hello\tWorld!\t2025\n";
    cout << "Backslash: This is \\ a backslash\n";
    cout << "Double Quote: He said, \"Hi!\\";
    cout << "Single Quote: It\'s sunny\n";
    cout << "Alert: \a (may not work on all systems)\n";
    cout << "Backspace: Hello\bWorld!\n";
}
```



```
cout << "Carriage Return: Hello\rWorld!\n";  
cout << "Form Feed: Hello\fWorld!\n";  
cout << "Vertical Tab: Hello\vWorld!\n";  
cout << "Question Mark: Why is this here \?\n";  
cout << "Null Character: This is\0null\n";  
return 0;  
}
```

Output

```
Newline: Hello  
World!  
Tab: Hello      World!  2025  
Backslash: This is \ a backslash  
Double Quote: He said, "Hi!"  
Single Quote: It's sunny  
Alert: (may not work on all systems)  
Backspace: HellWorld!  
World!ge Return: Hello  
Form Feed: Hello  
           World!  
Vertical Tab: Hello  
           World!  
Question Mark: Why is this here ?  
Null Character: This is
```

cin (Console Input):

`cin` reads data from the **standard input stream** (`stdin`). It uses the `>>` operator to extract input and stores it in variables.

- **Purpose:** Used to take input from the user via the console (standard input).
- **Syntax:** `cin >> variable;`
- The `>>` **operator**, called the "extraction operator," extracts data from the input stream and stores it in a variable.
- It stops reading input when it encounters whitespace, a newline, or an EOF.

Chaining Input

You can use the `>>` operator multiple times to read multiple inputs in a single line.

```
int a, b, c;  
cin >> a >> b >> c;
```

Handling Whitespace

- `cin` stops reading when it encounters whitespace (space, tab, or newline).
- If you want to read a full line (including spaces), use `cin.getline()`.

```
#include <iostream>
using namespace std;

int main() {
    char name[30];
    cout << "Enter your full name: ";
    cin.getline(name, 30); // Reads the entire line
    cout << "Hello, " << name << "!" << endl;
    return 0;
}
```

Error Handling in `cin`

If the user provides input of the wrong type (e.g., entering a string when an integer is expected), `cin` enters a **fail state** and stops working until the error is cleared. If the user provides invalid input, `cin` enters a fail state. Use the following to handle errors:

- `cin.clear()` : Clears the error flag.
- `cin.ignore(n, delimiter)` : Ignores up to `n` characters until a delimiter is encountered.

```
#include <iostream>
using namespace std;

int main() {
    int age;
    cout << "Enter your age: ";
    cin >> age;

    if (cin.fail()) { // Check if cin is in a fail state
        cout << "Invalid input! Please enter a number." << endl;
        cin.clear(); // Clear the error flag
        cin.ignore(1000, '\n'); // Ignore invalid input
    }
    return 0;
}
```

Output

```
Enter your age: q
Invalid input! Please enter a number.
```

Input and Output Redirection in C++

What is Redirection?

Redirection in C++ allows you to use files or other sources as input/output instead of the default **standard input (keyboard)** and **standard output (console)**.

- **Input Redirection (<)**: Reads input from a file instead of the keyboard.
- **Output Redirection (>)**: Writes output to a file instead of the console.

How to Use Redirection?

Command-Line Redirection Syntax

1. Input Redirection: `<`
 - Redirects the standard input (`cin`) to read from a file.
 - Example: `./program < input.txt`
2. Output Redirection: `>`
 - Redirects the standard output (`cout`) to write to a file.
 - Example: `./program > output.txt`
3. Append Output: `>>`
 - Appends the output to an existing file (instead of overwriting).
 - Example: `./program >> output.txt`
4. Combined Input and Output:
 - Redirect both input and output together.
 - Example: `./program < input.txt > output.txt`

Example Program

Program: Read from Input, Write to Output

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string name;
    int age;

    // Take input from user or redirected file
    cin >> name >> age;
```

```
// Output to console or redirected file
cout << "Name: " << name << endl;
cout << "Age: " << age << endl;

return 0;
}
```

Input File (input.txt)

```
John
30
```

Run the Program with Redirection

```
./program < input.txt > output.txt
```

Output File (output.txt)

```
Name: John
Age: 30
```

Key Notes on Redirection

- Default Behavior:**
 - `cin` reads from the keyboard.
 - `cout` writes to the console.
- With Redirection:**
 - Input (`cin`) reads from the specified input file.
 - Output (`cout`) writes to the specified output file.
- Overwrite vs Append:**
 - `>` overwrites the file if it exists.
 - `>>` appends to the file without overwriting.
- Error Redirection:**
 - Redirect the standard error (`cerr`) to a file using `2>` .
 - Example: `./program 2> error.txt`

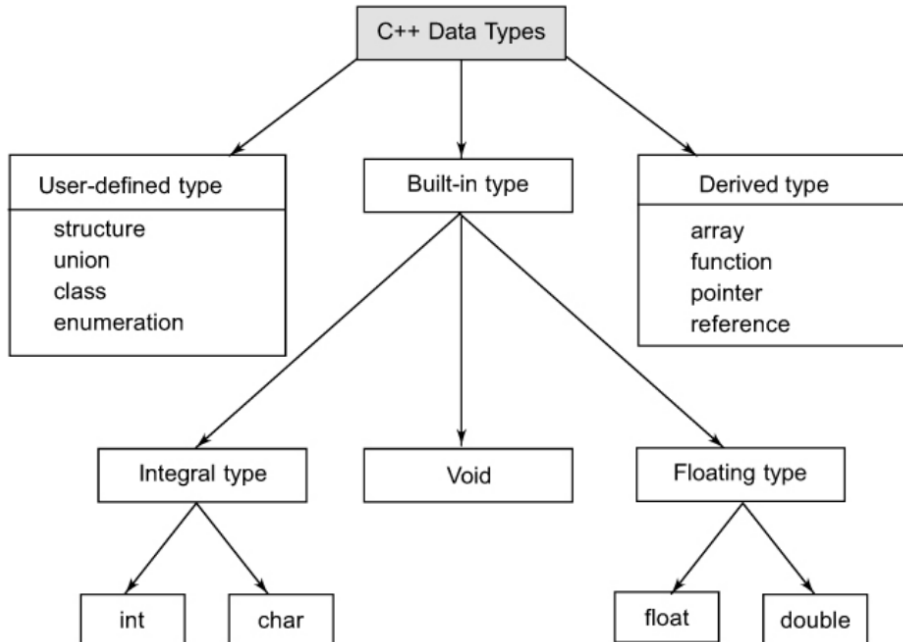
Symbol	Description	Example Command
<	Redirect standard input from a file	<code>./program < input.txt</code>
>	Redirect standard output to a file	<code>./program > output.txt</code>
>>	Append standard output to a file	<code>./program >> output.txt</code>

Symbol	Description	Example Command
2>	Redirect standard error to a file	<code>./program 2> error.txt</code>
2>>	Append standard error to a file	<code>./program 2>> error.txt</code>
< & >	Combine input and output redirection	<code>./program < input.txt > output.txt</code>

Datatypes

Data types simply refers to the type and size of data associated with variables and functions.

The Following diagram shows types C++.



C++ data types are broadly categorized into:

1. User-defined Types

Custom types defined by the programmer:

- **struct** – Group of related variables.
- **union** – Memory-efficient structure storing only one member at a time.
- **class** – Object-oriented construct containing data and methods.
- **enum** – Defines a set of named integer constants.

2. Built-in Types

Predefined primitive data types:

- **Integral Type**
 - `int` – Integer numbers
 - `char` – Character data
- **Floating Type**
 - `float` – Single-precision floating-point

- `double` – Double-precision floating-point
- **Void**
 - Represents absence of value, used in functions with no return.

Basic Data Types in C++

a. `int` (Integer)

- Used to store whole numbers.
- Size: Usually 4 bytes (platform-dependent).
- Example:

```
int age = 25;
```

b. `char` (Character)

- Stores a single character.
- Size: 1 byte.
- Stored as ASCII value.
- Example:

```
char grade = 'A';
```

c. `float` (Floating-point number)

- Stores decimal numbers with single precision.
- Size: Usually 4 bytes.
- Example:

```
float temperature = 36.6f;
```

d. `double` (Double-precision floating-point)

- Used for more precision than `float`.
- Size: Usually 8 bytes.
- Example:

```
double pi = 3.1415926535;
```

e. `bool` (Boolean)

- Represents logical values: `true` or `false`.
- Size: Usually 1 byte.
- Example:

```
bool isRaining = false;
```

f. `void`

- Used to indicate **no return type** in functions.
- Cannot define variables of type `void`.
- Example:

```
void displayMessage() {
    cout << "Hello, World!";
}
```

Size and Range of Basic Data Types (Typical on 32/64-bit systems)

Data Type	Size (bytes)	Range
<code>char</code>	1	-128 to 127 (signed), 0 to 255 (unsigned)
<code>bool</code>	1	<code>true</code> or <code>false</code>
<code>int</code>	4	-2,147,483,648 to 2,147,483,647
<code>float</code>	4	~±3.4e±38 (6-7 digits precision)
<code>double</code>	8	~±1.7e±308 (15 digits precision)

3. Derived Types

Formed using built-in or user-defined types:

- `array` – Collection of fixed number of elements
- `function` – Reusable block of code
- `pointer` – Holds memory address
- `reference` – Alternative name for a variable

C++ Data Type Modifiers

C++ provides **type modifiers** to alter the **range and storage size** of basic data types (except `void`) to suit different computational needs.

Applicable Data Types

Modifiers can be applied to:

- `int` and `char` (primarily)
- `double` (only `long` is applicable)
- `void` does **not support any modifier**.

List of Modifiers

1. `signed`

- Default for `int` and `char` (unless explicitly stated otherwise)
- Allows both **positive and negative values**
- Example:

```
signed int x = -100;  
signed char ch = 'A';
```

2. `unsigned`

- Restricts values to **only positive numbers (including 0)**
- Increases the upper limit of the range
- Example:

```
unsigned int x = 100;  
unsigned char ch = 255;
```

3. `short`

- Reduces the **size** of `int` (usually 2 bytes)
- Used when memory saving is critical
- Example:

```
short int x = 32767;
```

4. `long`

- Increases the **storage size** of the data type
- May also be applied to `double` for higher precision
- Examples:

```
long int x = 9223372036854775807;  
long double pi = 3.141592653589793238;
```

Combinations of Modifiers with `int`

Modifier Combination	Typical Size	Value Range
<code>short int</code>	2 bytes	-32,768 to 32,767
<code>unsigned short int</code>	2 bytes	0 to 65,535
<code>int</code> / <code>signed int</code>	4 bytes	-2,147,483,648 to 2,147,483,647
<code>unsigned int</code>	4 bytes	0 to 4,294,967,295
<code>long int</code>	4 or 8 bytes	Wider range, platform-dependent
<code>unsigned long int</code>	4 or 8 bytes	Wider positive-only range

Note

- Modifiers help fine-tune memory and performance based on program requirements.
- Always consider the platform/compiler as **data type sizes may vary**.

Derived Datatypes

In C++, **derived data types** are data types that are built from the fundamental (basic) data types. They allow us to group basic data or refer to it in more complex ways. Common derived types include **arrays**, **functions**, **pointers**, and **references**. These help manage collections of data, reuse code, and manipulate memory efficiently in C++.

Arrays

An **array** is a collection of elements **of the same type**, stored one after another in memory.

Example:

```
int numbers[3];
numbers[0] = 10;
numbers[1] = 20;
numbers[2] = 30;
cout << "The second element is " << numbers[1] << endl;
```

Arrays are data structures that store sets of data items under a single name declaration. Each element of the array must be of the same type. When allocated, an array takes up a contiguous block of memory. The elements can be accessed via an index (a non-negative integer).

Declaring Arrays

An array is declared like any other variable (<type> then <variable name>) but also included is the number of items in the array (the size must be specified when declaring an array). When an array is declared, memory is allocated for the array, and it remains that size. Each element is accessed by an index (or subscript), beginning with the zero-ith element. So, for example, if you need to store a set of 5 integer grades, you can declare an array like the following:

```
int grades[5];
```

where the array will have space for 5 integers indexed from 0-4: grades[0], grades[1], grades[2], grades[3], & grades[4] each one can contain an integer. If you need to access the 3rd item, you would access grades[2] or grades[n-1] where n is the total number of elements in the array. In memory, you can picture it looking something like the following:

	grades
0	
1	
2	
3	
4	

- Once allocated, this allows you to have one variable name for a set of same-type items.
- You can use the entire array as a single object, or each item in the array individually, as shown below.

An individual array element can be used anywhere that a normal variable can be used. For example, if you want to assign the 3rd element the value of 85, then you would write

```
grades[2] = 85;
```

Now our array would look something like this:

	grades
0	
1	
2	85
3	
4	

Or, if you have an integer variable called `g5` and you want to assign to it the value of the 5th element (assuming it has a value), you would write

```
g5 = grades[4];
```

which then gives `g5` the value of whatever that 5th element is in the array.

Arrays make it much easier for handling large data sets without having to declare each of the items as individual variables. They also allow you to perform subtasks, like filling the elements of an array from user input or a file, displaying all of the elements of an array in forward or reverse order, sorting the values of an array in ascending or descending order, determining the sum, average, or some other statistic from the array values, and finding the highest or lowest value.

With a for loop, for example, you can easily access each item in a large array. Finding the lowest value, or adding up all the values is much easier to implement using an array.

The following loop will sum up all the values in an array of 100 items:

```
for (int i = 0; i < 100; ++i)
    sum += grades[i];
```

Remember to start with zero or you will miss the first item.

TIP: When specifying the size of an array, you can use a named constant or `#defined` value, such as with the following:

```
#include<iostream>
const int MAX_SZ = 50;
int main()
{
    int grades[MAX_SZ];
}
```

OR

```
#include<iostream>
#define MAX_SZ 50;
int main()
{
    int grades[MAX_SZ];
}
```

This is helpful for a couple of reasons. First, it improves program readability. It also makes the program more versatile and maintainable because you can change any reference to the size of the array in one easy to find location.

Initializing Arrays

If the initial values of an array are known, the elements in the array can be initialized when declared, such as:

```
int counters[5] = { 0, 0, 0, 1, 5 };
```

The above line of code sets the value of `counters[0]` to 0, `counters[1]` to 0, `counters[2]` to 0, `counters[3]` to 1, and `counters[4]` to 5. This would be equivalent to the following 6 lines of code:

```
int counters[5];
counters[0] = 0;
counters[1] = 0;
counters[2] = 0;
```

```
counters[3] = 1;
counters[4] = 5;
```

An array of characters can be initialized in a similar manner:

```
char vowels[5] = { 'a', 'e', 'i', 'o', 'u' };
```

It's not necessary to completely initialize the entire array. If the first few are initialized, the remaining are set to zero (0). Consider the following:

```
float sample_data[500] = { 100.0, 300.0, 500.5 };
```

The first 3 elements are initialized to the values provided; the remaining 497 elements will be set to zero (0). However, no assumptions may be made about the values of the elements of an uninitialized array (i.e. they are NOT zero (0) by default).

If all the elements need to be initialized to something other than zero (0), the best way to do that is to use a for loop.

```
int array_values[10];
int i;
for ( i = 0; i < 10; ++i )
    array_values[i] = i * i;
```

This for loop initializes each element to the square of the element number (subscript number), so the array will contain 0, 1, 4, 9, 16, 25, 36, 49, 64, 81 after initialization.

C++ allows you to declare an array without specifying the number of elements only if you initialize every element of the array when it is declared. The following:

```
int counters[] = { 0, 0, 0, 1, 5 };
```

will implicitly dimension the array to 5 elements.

You can also use the index numbers when initializing. So if you know specific elements need to be initialized to a certain value, you can do the following:

```
float sample_data[] = { [0] = 1.0, [49] = 100.0, [99] = 200.0 };
```

Because the largest index number specified above is 99, sample_data will be set to contain 100 elements; the remaining uninitialized elements initialized to zero (0).

Functions

Definition: A **function** is a reusable block of code that performs a specific task.

Example:

```
int add(int x, int y) {  
    return x + y;  
}  
int result = add(3, 4);  
cout << "3 + 4 = " << result << endl;
```

Pointers

Definition: A **pointer** is a variable that stores the **memory address** of another variable.

Example:

```
int number = 42;  
int *ptr = &number;  
cout << "Address stored in ptr: " << ptr << endl;  
cout << "Value pointed to by ptr: " << *ptr << endl;
```

References

Definition: A **reference** is an alternate name for an existing variable.

Example:

```
int original = 100;  
int &alias = original;  
alias = 200;  
cout << "original: " << original << endl;  
cout << "alias: " << alias << endl;
```

Functions, Call by Reference

What is a Function?

A **function** is a reusable block of code that performs a specific task. Functions help organize code, promote reusability, improve readability, and allow modular programming.

Syntax

```
return_type function_name(parameter_list) {  
    // body of function  
}
```

Example:

```
int add(int a, int b) {  
    return a + b;  
}
```

Function Types

- **User-defined functions:** Written by the programmer.
- **Library functions:** Provided by C++ libraries (e.g. `sqrt()` , `abs()`).

Function Declaration vs Definition

Declaration

```
int add(int, int);
```

Tells the compiler a function exists.

Definition

```
int add(int a, int b) {  
    return a + b;  
}
```

Contains actual code. Declaration not required if defined before use.

Function Parameters

When calling a function in C++, values are passed to parameters. The way values are passed determines whether the function can **modify** the original variable or just work on a **copy**.

Call by Value

- A **copy** of the actual value is passed.
- Changes made inside the function **do not affect** the original variable.

Syntax:

```
void modify(int x) {  
    x = x + 5;  
}
```

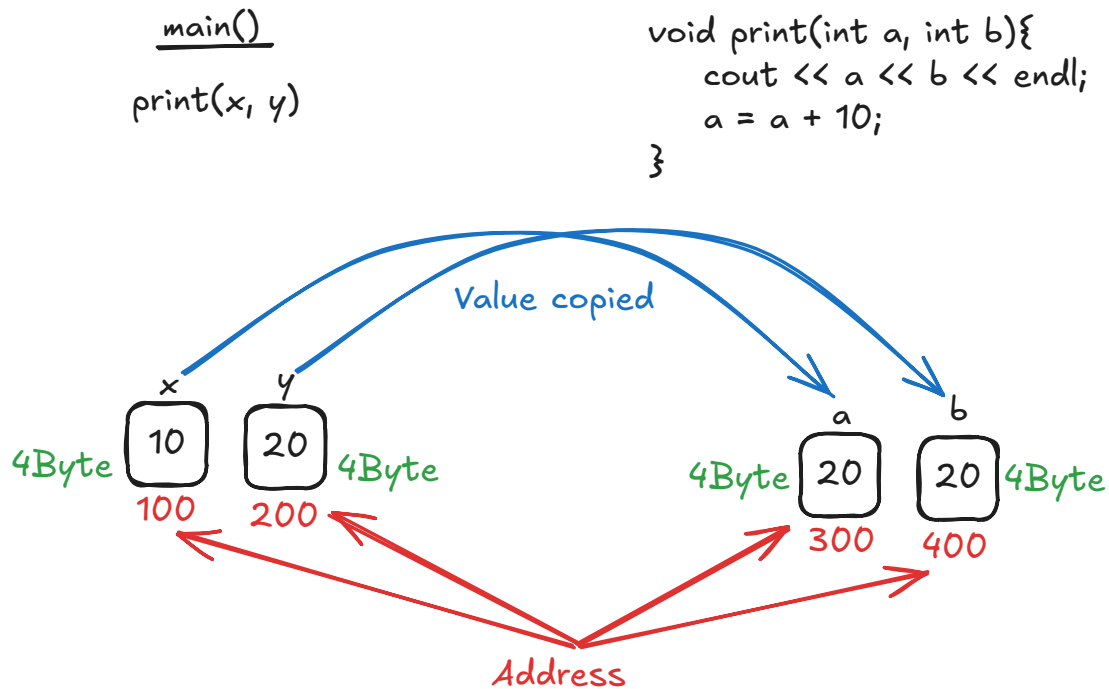
Example

```
int main() {  
    int a = 10;  
    modify(a);  
    cout << a; // Output: 10 (unchanged)  
}
```

Note

When original value should remain unchanged.

Call by Value



Call by Address (Using Pointers)

- The function receives the **address** of the variable.
- Modifying the dereferenced pointer changes the original value.

Syntax

```
void modify(int *x) {  
  *x = *x + 5;  
}
```

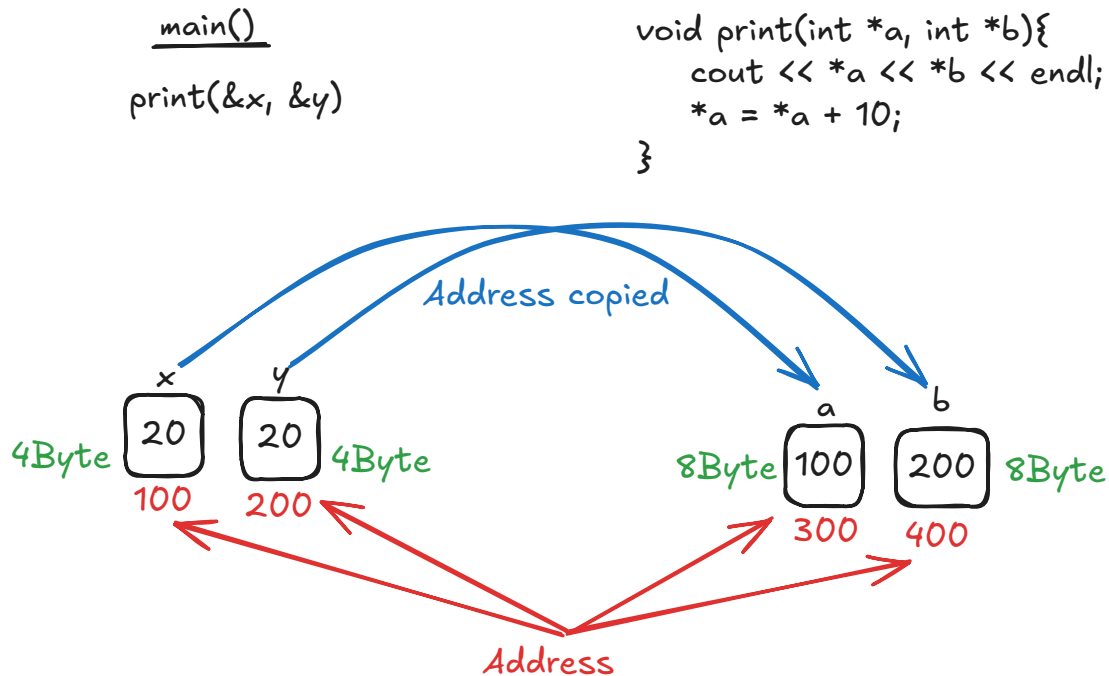
Example

```
int main() {  
  int a = 10;  
  modify(&a);  
  cout << a; // Output: 15 (modified)  
}
```

Note

- When you want to modify the original variable.
- Used in dynamic memory management and low-level coding.

Call by Address



Call by Reference

- A **reference** (alias) of the original variable is passed.
- Changes inside the function affect the original variable **directly**.

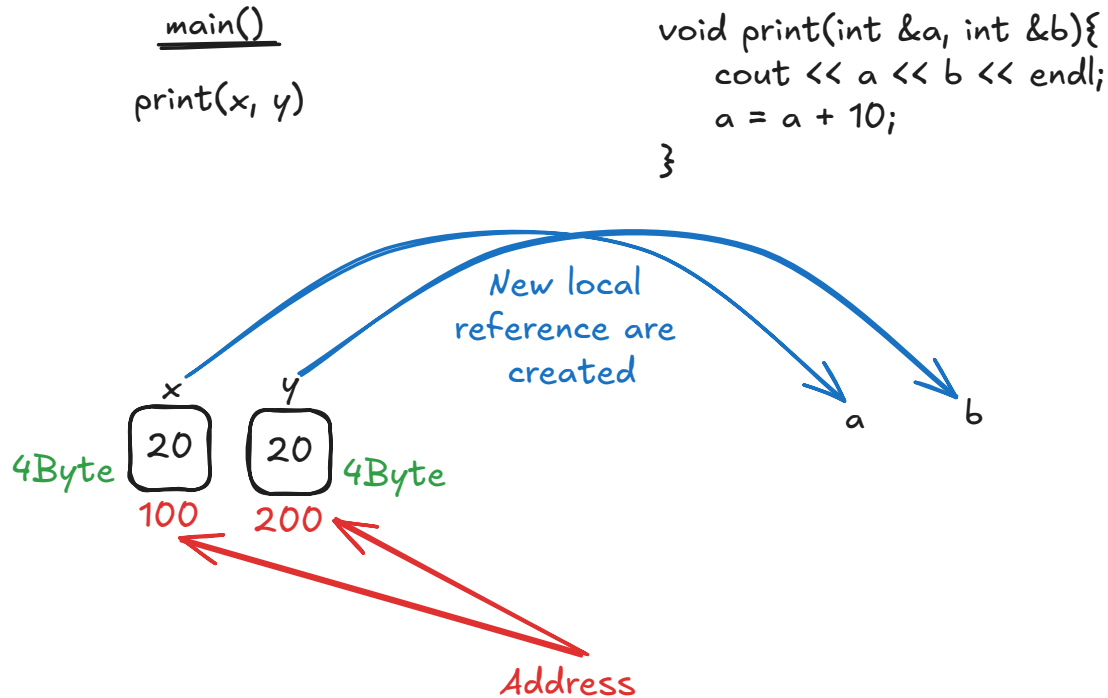
Syntax

```
void modify(int &x) {  
  x = x + 5;  
}
```

Example

```
int main() {  
  int a = 10;  
  modify(a);  
  cout << a; // Output: 15 (modified)  
}
```

Call by Reference



Default Arguments

Allow parameters to have default values.

```
void greet(string name, string prefix = "Mr.") {  
    cout << prefix << " " << name;  
}
```

Call:

```
greet("John");           // uses "Mr."  
greet("Jane", "Dr.");    // uses "Dr."
```

- Must be trailing parameters.
- Declared once, typically in prototype.

Const Arguments

Prevents function from modifying input.

```
void print(const string &msg) {  
    cout << msg;
```

```
}
```

- Use with references for safety and performance.
- Enables passing const and temporary objects.

Function Overloading

Function overloading means having **more than one function** with the **same name** in the same scope, but each has **different**:

- Number of parameters
- Type of parameters
- Order of parameters

Syntax

```
int add(int a, int b);           // 2 int parameters
float add(float a, float b);     // 2 float parameters
int add(int a, int b, int c);    // 3 int parameters
```

Important

The correct version is selected based on **arguments passed** during the function call — this process is known as **compile-time polymorphism**.

Example

```
#include <iostream>
using namespace std;

int add(int a, int b) {
    return a + b;
}

float add(float a, float b) {
    return a + b;
}

int add(int a, int b, int c) {
    return a + b + c;
}

int main() {
    cout << add(3, 4) << endl;           // Calls int version
    cout << add(3.5f, 2.5f) << endl;     // Calls float version
}
```

```
cout << add(1, 2, 3) << endl;    // Calls 3-parameter version
}
```

Output:

```
7
6
6
```

Note

- Functions must differ in **parameter list**.
- Return type **alone** cannot distinguish overloaded functions.

Invalid Example:

```
int show(int a);
float show(int a); // ✗ Error: Only return type differs
```

Recursion

What is Recursion?

Recursion is a programming technique where a function calls itself to solve a problem. It breaks down a complex problem into smaller instances of the same problem.

Structure of a Recursive Function

Every recursive function must include:

1. **Base Case** – condition to stop recursion.
2. **Recursive Case** – function calls itself with modified parameters.

```
void recursiveFunction() {
    if (base_case_condition)
        return; // base case
    else
        recursiveFunction(); // recursive call
}
```

Example: Factorial

```
int factorial(int n) {  
    if (n <= 1)  
        return 1; // base case  
    else  
        return n * factorial(n - 1); // recursive call  
}
```

Example: Fibonacci

```
int fibonacci(int n) {  
    if (n <= 1)  
        return n;  
    else  
        return fibonacci(n - 1) + fibonacci(n - 2);  
}
```

Common Issues

- Forgetting the base case (causes infinite recursion).
- Incorrect base case condition.
- Not reducing the problem size.

Class Concept

Class in C++

- In C++, a **class** is a blueprint for creating objects.
- It encapsulates data (variables) and functions (methods) that operate on that data into a single unit.

Key Components of a Class

1. **Data Members:** Variables that hold the data of the class.
2. **Member Functions:** Functions that define the behavior of the class.
3. **Access Specifiers:** Control the access to the class members. Common specifiers:
 - **public:** Members are accessible from outside the class.
 - **private:** Members are accessible only within the class.
 - **protected:** Members are accessible within the class and derived classes.

Class Declaration

A class in C++ is declared using the `class` keyword. It serves as a blueprint for creating objects and defines the properties (data members) and behaviors (member functions).

Syntax:

```
class <ClassName> {  
    // Access Specifiers  
    // Data Members  
    // Member Functions  
};
```

Object Creation

Objects are instances of a class. Each object gets its own copy of the class's data members.

Syntax:

```
<ClassName> <ObjectName>;
```

Here, `ClassName` is the name of the class, and `ObjectName` is the name of the object.

Accessing Members

You can access class members using:

- The **dot operator** (.) for accessing members of an object.
- The **arrow operator** (->) when accessing members through a pointer to the object.

Access Syntax with dot operator

- **Data Members**

```
objectName.dataMember;
```

- **Member Functions:**

```
objectName.memberFunction();
```

Access with Pointers:

- **Data Members:**

```
pointerToObject->dataMember;
```

- **Member Functions:**

```
pointerToObject->memberFunction();
```

Purpose	Syntax Example
Create an object	ClassName objectName;
Access data member	objectName.dataMember;
Access member function	objectName.memberFunction();
Access data member with pointer	pointerToObject->dataMember;
Access function with pointer	pointerToObject->memberFunction();

Data Members

These are variables that store the attributes of a class. Each object of the class will have its own copy of the data members.

Example

```
class Car {  
    string brand; // Data member
```

```
int speed;    // Data member
};
```

Member Functions

Member functions define the behavior of the class. They operate on the data members of the class.

Syntax:

- Defined **inside the class**:

```
class Car {
    void start() {
        cout << "Car started!" << endl;
    }
};
```

- Defined **outside the class**:

```
class Car {
    void start(); // Function prototype
};

void Car::start() {
    cout << "Car started!" << endl;
}
```

Access Specifiers

Access specifiers control the visibility of class members. There are three types:

- public** : Members declared as **public** can be accessed from outside the class.
- private** : Members declared as **private** can only be accessed within the class. This ensures encapsulation and data hiding. The members of a **class** in C++ are **private by default**.
- protected** : Members declared as **protected** can be accessed in derived (inherited) classes and the class itself.

Syntax

```
class Car {
private:
    int speed; // Only accessible within the class
};
```

```
protected:
    int fuel; // Accessible in the class and derived classes

public:
    string brand; // Accessible from outside the class
};
```

public

- Members declared as `public` are **accessible from outside the class**.
- Public members are generally used for interfaces to interact with objects of the class.

Example :

```
#include <iostream>
using namespace std;

class Car {
public: // Public access specifier
    string brand; // Public data member
    void display() { // Public member function
        cout << "Brand: " << brand << endl;
    }
};

int main() {
    Car car1;
    car1.brand = "Toyota"; // Accessing public data member
    car1.display(); // Accessing public member function
    return 0;
}
```

Output

```
Brand: Toyota
```

private

- Members declared as `private` are **only accessible within the class**. They cannot be accessed directly from outside the class.
- This is the **default access specifier** if no specifier is provided.
- Private members are typically used to store sensitive or internal data, ensuring **encapsulation**.

```

#include <iostream>
using namespace std;

class Car {
private: // Private access specifier
    int speed; // Private data member

public:
    void setSpeed(int s) { // Public member function to set speed
        if (s > 0)
            speed = s;
        else
            cout << "Invalid speed!" << endl;
    }
    int getSpeed() { // Public member function to get speed
        return speed;
    }
};

int main() {
    Car car1;
    // car1.speed = 100; // Error: 'speed' is private
    car1.setSpeed(120); // Indirect access through public function
    cout << "Speed: " << car1.getSpeed() << " km/h" << endl;
    return 0;
}

```

protected

- Members declared as `protected` are **accessible within the class itself and in derived classes**.
- They are not accessible directly from outside the class.
- Protected members are used when designing **inheritance** hierarchies, where child classes can access certain members of the parent class.

Access Specifier	Access Within Class	Access in Derived Class	Access Outside Class	
public	✓ Yes	✓ Yes	✓ Yes	
protected	✓ Yes	✓ Yes	✗ No	
private	✓ Yes	✗ No	✗ No	

Type Definition

What is `typedef` and Why Use It?

- The keyword `typedef` (short for "type definition") in C++ is used to create an alias for an existing data type.
- **Why use `typedef` ?**
 - Using a type alias can make code more readable and easier to maintain.
 - **Simplify complex types:** Long or nested types (like function pointers or templates) become easier to work with by giving them a short name.
 - **Improve code readability:** A well-chosen alias name (e.g. `Pixel` for an `int` representing a screen pixel) makes the code self-documenting and clearer in intent.
 - **Ease maintenance:** Changing the underlying type (for example, for portability or new requirements) can be done by editing the `typedef`, without touching the rest of the code.
 - **Maintain C compatibility:** Many C libraries and legacy code use `typedef` extensively (e.g. for struct names and pointer types), so understanding it helps in interfacing with such code.

NB:

a `typedef` *does not create a new type*, it simply creates a synonym for an existing type. The alias is treated by the compiler exactly as if you had written the original type

Basic Syntax and Examples

The basic syntax of a `typedef` declaration is:

```
typedef existing_type AliasName;
```

```
`#include <iostream>
// Define a typedef alias "Integer" for the type int
typedef int Integer;
int main() {
    Integer count = 5;
    // 'count' is an int, via alias
    std::cout << "count = " << count << std::endl;
```

```
    return 0;
}
```

In this example, `Integer` is an alias for `int`. Declaring `Integer count` is exactly the same as declaring `int count`. The compiler treats `Integer` as if it were `int` whenever it sees it.

Using `typedef` with Structures (Structs)

- One common use of `typedef` in C++ is with structures pointer.
- Though C++ treats structure as a datatype we don't need to use `struct` key word.

```
// C++ style struct (no typedef needed)
struct Point {
    int x;
    int y;
};

Point p1;           // OK in C++ (Point is a usable type name)
```

- However in case of pointer or complex pointers we can use `typedef` to make more sense of datatype.

```
struct Node {
    int data;
    Node* next;
};
typedef struct Node* NodePtr; // NodePtr is an alias for "pointer to Node"

Node n;
NodePtr head = &n; // head is a Node*
```

Now `NodePtr` can be used wherever a pointer to `Node` is needed, simplifying declarations. (We'll discuss pointers more in the next section.)

```
class Rectangle {
public:
    int width, height;
    int area() { return width * height; }
};

// Create a type alias
typedef Rectangle Rect;
```

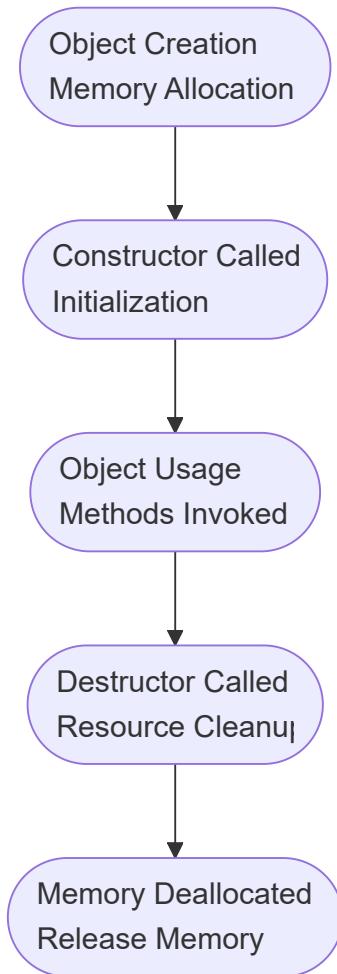
```
int main() {  
    Rect r1;  
    r1.width = 10;  
    r1.height = 5;  
    std::cout << "Area: " << r1.area() << std::endl;  
}
```

The compiler treats `Rect` exactly the same as `Rectangle` .

Objects, Constructors and Destructors

Object and Constructors

The **object lifecycle** in C++ describes the various stages an object goes through during its existence, from creation to destruction. These stages are defined by how memory is allocated, the constructor is called, the object is used, and eventually destroyed when the destructor is called.



Stages of Object Lifecycle

- 1. Object Declaration:**
 - The process of defining an object in code. Memory is allocated for the object when declared.
- 2. Constructor Call:**
 - When the object is created, the constructor initializes the object. If no constructor is explicitly defined, the default constructor is used.
- 3. Object Usage:**

- The object performs its intended functionality through methods and member functions. During this stage, its state can change.

4. Destructor Call:

- When the object goes out of scope or is explicitly deleted, the destructor is called to clean up resources, releasing allocated memory or performing other final tasks.

5. Memory Deallocation:

- After the destructor finishes execution, the memory allocated to the object is released.

Constructors

- A **constructor** is a special member function in C++ that is automatically called when an object of a class is created. It is primarily used to initialize the object's data members and perform any setup required for the object.

Key Features of a Constructor:

1. **Same Name as the Class:** The constructor's name must match the name of the class.
2. **No Return Type:** Constructors do not have a return type, not even `void`.
3. **Automatically Called:** It is invoked automatically when an object is created.
4. **Can Be Overloaded:** A class can have multiple constructors (constructor overloading) with different parameter lists.

General Syntax of a Constructor

Here's the general syntax for defining a constructor in a class:

```
class ClassName {  
public:  
    ClassName() {  
        // Initialization code  
    }  
};
```

Types of Constructors

1. Default Constructor

A **default constructor** is a special type of constructor in C++ that **takes no arguments** or **has all default arguments**. It is automatically invoked when an object of a class is created **without passing any arguments**. Its primary purpose is to initialize an object with default values.

Key Characteristics of a Default Constructor

1. **No Parameters:** A default constructor has no parameters or has parameters with default values.
2. **Automatic Invocation:** It is automatically called when an object is created.
3. **Implicitly Defined:** If no constructor is explicitly defined in a class, the compiler provides an implicit default constructor.
4. **User-Defined Option:** A programmer can define a custom default constructor to initialize members with specific values.
5. **Initialization Purpose:** Ensures that the object starts in a valid state.

Example

```
#include <iostream>
using namespace std;
class MyClass {
public:
    MyClass() { // Default constructor
        cout << "Default constructor called!" << endl;
    }
};
int main() {
    MyClass obj; // Default constructor is automatically called
    return 0;
}
```

Implicit Default Constructor

If you don't define any constructor in your class, the compiler generates a default constructor automatically.

- It does nothing explicitly.
- The data members of built-in types are left uninitialized.
- If a class has no constructors, the compiler automatically generates a default constructor.

```
class MyClass {
    // Compiler provides an implicit default constructor
};
```

Explicit Default Constructor

You can explicitly define a default constructor to control the initialization.

```
class MyClass {  
public:  
    MyClass() {  
        // Initialization code  
    }  
};
```

Benefits of a Default Constructor

1. **Automatic Initialization:** It ensures that objects start with valid initial values, preventing undefined behavior.
2. **Ease of Use:** Simplifies object creation when no specific initialization is required.
3. **Supports Polymorphism:** Plays a crucial role in initializing base and derived class objects.
4. **Compiler-Generated Simplicity:** Eliminates the need for explicit initialization in simple classes.

When Is a Default Constructor Needed?

1. When you create objects without arguments:

```
MyClass obj;
```

2. When working with **arrays of objects**:

```
MyClass arr[5]; // Requires a default constructor
```

3. When creating **dynamic objects**:

```
MyClass* obj = new MyClass(); // Default constructor is called
```

4. When using **inheritance**:

- A default constructor ensures proper initialization of base and derived classes.

2. Parameterized Constructor

A **parameterized constructor** is a constructor in C++ that takes arguments. It is used to initialize objects with specific values provided during the object creation, making it more flexible and allowing customized initialization.

Key Features of Parameterized Constructors:

1. **Takes Parameters:**

- Unlike a default constructor, a parameterized constructor accepts arguments to initialize data members.

2. Called During Object Creation:

- When an object is created with arguments, the parameterized constructor is automatically invoked.

3. Constructor Overloading:

- Parameterized constructors can be part of constructor overloading, where multiple constructors exist with different parameter lists.

4. Initialization:

- Typically used to initialize member variables with specific values or perform complex initialization.

Example

```
#include <iostream>
using namespace std;

class MyClass {
    int a, b;
public:
    // Parameterized Constructor
    MyClass(int x, int y) {
        a = x;
        b = y;
    }

    void display() {
        cout << "a = " << a << ", b = " << b << endl;
    }
};

int main() {
    MyClass obj(10, 20); // Parameterized constructor is called
    obj.display();       // Output: a = 10, b = 20
    return 0;
}
```

Advantages of Parameterized Constructor

1. Flexible Initialization:

- Allows different objects of the same class to be initialized with different values.

2. Avoids Manual Initialization:

- Simplifies initialization by encapsulating it within the constructor.

3. Enhances Code Readability:

- Using parameterized constructors makes the code more concise and easier to maintain.

Error Example

```
#include <iostream>
using namespace std;

class MyClass {
    int a, b;
public:
    // Parameterized Constructor
    MyClass(int x, int y) {
        a = x;
        b = y;
    }

    void display() {
        cout << "a = " << a << ", b = " << b << endl;
    }
};

int main() {
    MyClass obj(10, 20); // Parameterized constructor is called
    MyClass obj_err; // Error
    obj.display();      // Output: a = 10, b = 20
    return 0;
}
```

As the user defined Parameterized constructor compiler will not add its default constructor. Hence, user defined default constructor needs to be added also.

```
#include <iostream>
using namespace std;

class MyClass {
    int a, b;
public:
    // Default constructor
    MyClass() {
        a = 0;
        b = 0;
    }
    // Parameterized Constructor
    MyClass(int x, int y) {
        a = x;
    }
}
```

```

        b = y;
    }

    void display() {
        cout << "a = " << a << ", b = " << b << endl;
    }
};

int main() {
    MyClass obj(10, 20); // Parameterized constructor is called
    MyClass obj_err; // Error
    obj.display();      // Output: a = 10, b = 20
    return 0;
}

```

Ambiguity in Constructor Calls:

- If multiple constructors exist with similar parameter lists, ambiguities may arise. Use distinct parameter types to avoid conflicts.

3. Constructor with Default Arguments

A **constructor with default arguments** is a constructor that allows you to provide **default values** for some or all of its parameters. This means if an object is created without providing certain arguments, the constructor will automatically use the default values.

Key Features

1. Default Values:

- You specify default values in the constructor declaration.
- If an argument is omitted when creating an object, the default value is used.

2. Combines Flexibility and Code Simplicity:

- Instead of writing multiple overloaded constructors, you can use one constructor with default arguments.

3. Order of Parameters:

- Default arguments must be specified from **right to left**. Non-default arguments should always precede default ones.

Syntax

```

class ClassName {
public:
    ClassName(int a = 0, int b = 0); // Constructor with default

```

```
arguments  
};
```

Example

```
#include <iostream>  
using namespace std;  
  
class MyClass {  
    int a, b;  
public:  
    MyClass(int x = 0, int y = 0) { // Default arguments  
        a = x;  
        b = y;  
        cout << "Constructor called with a = " << a << " and b = " << b  
        << endl;  
    }  
};  
  
int main() {  
    MyClass obj1;           // Uses default values (a = 0, b = 0)  
    MyClass obj2(10);       // Uses a = 10, b = 0  
    MyClass obj3(10, 20);   // Uses a = 10, b = 20  
    return 0;  
}
```

Destructor in C++

What is a Destructor?

A **destructor** is a special member function in C++ that is called automatically when an object of the class goes out of scope or is explicitly deleted. Its primary purpose is to release resources (like dynamic memory, file handles, etc.) allocated during the object's lifetime.

Key Features of a Destructor

- 1. Same Name as the Class:**
 - The destructor has the same name as the class, prefixed with a tilde (~).
- 2. No Parameters and No Return Type:**
 - A destructor cannot have parameters or a return type.
- 3. Automatic Invocation:**
 - It is invoked automatically when the object is destroyed.
- 4. One Destructor Per Class:**
 - A class can have **only one destructor**.

5. Used for Cleanup:

- Primarily used to free memory, close files, release resources, etc.

Syntax

```
class ClassName {
public:
    ~ClassName() {
        // Code for cleanup
    }
};
```

Example

```
#include <iostream>
using namespace std;

class MyClass {
public:
    MyClass() {
        cout << "Constructor called!" << endl;
    }

    ~MyClass() {
        cout << "Destructor called!" << endl;
    }
};

int main() {
    MyClass obj; // Constructor is called
    cout << "Object in use..." << endl;
    return 0; // Destructor is called when `obj` goes out of scope
}
```

When is a Destructor Called?

1. When an object goes out of scope:
 - For example, at the end of a function or block.
2. When `delete` is used for a dynamically allocated object:

```
MyClass* obj = new MyClass(); // Constructor called
delete obj; // Destructor called
```

3. When the program ends and global/static objects are destroyed.

 Note

- A class can only have **one destructor**.
- Use destructors to release resources and avoid memory leaks.

Enumerated Datatype

1. Enumerated Data Type (enum)

Definition:

An **enumerated data type** is a user-defined type consisting of integral constants, giving meaningful names to values.

Syntax:

```
enum EnumName { Value1, Value2, ..., ValueN };
```

Example:

```
enum Color { RED, GREEN, BLUE };  
Color c = GREEN;
```

Internal Behavior:

- Each enumerator is assigned an integer value starting from 0 by default.
- RED = 0, GREEN = 1, BLUE = 2
- Custom values can be assigned:

```
enum Color { RED = 1, GREEN = 3, BLUE = 5 };
```

```
#include <iostream>  
using namespace std;  
  
enum Day {  
    Sunday,  
    Monday,  
    Tuesday,  
    Wednesday,  
    Thursday,  
    Friday,  
    Saturday  
};  
  
int main() {  
    Day today = Wednesday;  
  
    if (today == Wednesday) {  
        cout << "It's midweek!" << endl;  
    }  
}
```

```

}

// Print enum values
cout << "Enum values:" << endl;
cout << "Sunday = " << Sunday << endl;
cout << "Monday = " << Monday << endl;
cout << "Tuesday = " << Tuesday << endl;
cout << "Wednesday = " << Wednesday << endl;
cout << "Thursday = " << Thursday << endl;
cout << "Friday = " << Friday << endl;
cout << "Saturday = " << Saturday << endl;

return 0;
}

```

Output :

```

It's midweek!
Enum values:
Sunday = 0
Monday = 1
Tuesday = 2
Wednesday = 3
Thursday = 4
Friday = 5
Saturday = 6

```

Notes

- By default, `enum` assigns **integer values starting from 0**.
- You can customize the values like: `Sunday = 1, Monday = 2, ...`
- `enum` improves code **readability** and **type safety**.

2. Enumerated Class (`enum class`)

Definition:

Introduced in C++11, `enum class` improves type safety and avoids polluting the global namespace.

Syntax:

```
enum class EnumName { Value1, Value2, ..., ValueN };
```

Example:

```
enum class Direction { LEFT, RIGHT, UP, DOWN };  
Direction d = Direction::LEFT;
```

```
#include <iostream>  
using namespace std;  
  
enum class Color {  
    Red = 1,  
    Green = 2,  
    Blue = 4  
};  
  
int main() {  
    Color favorite = Color::Green;  
  
    // Output enum value by casting to int  
    cout << "Favorite color value: " << static_cast<int>(favorite) <<  
endl;  
  
    // Using switch-case with enum class  
    switch (favorite) {  
        case Color::Red:  
            cout << "You chose Red." << endl;  
            break;  
        case Color::Green:  
            cout << "You chose Green." << endl;  
            break;  
        case Color::Blue:  
            cout << "You chose Blue." << endl;  
            break;  
    }  
  
    return 0;  
}
```

Output :

```
Favorite color value: 2  
You chose Green.
```

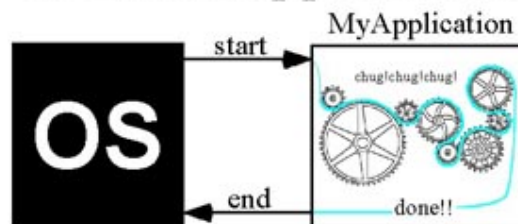
GUI Basics

GUI Basics

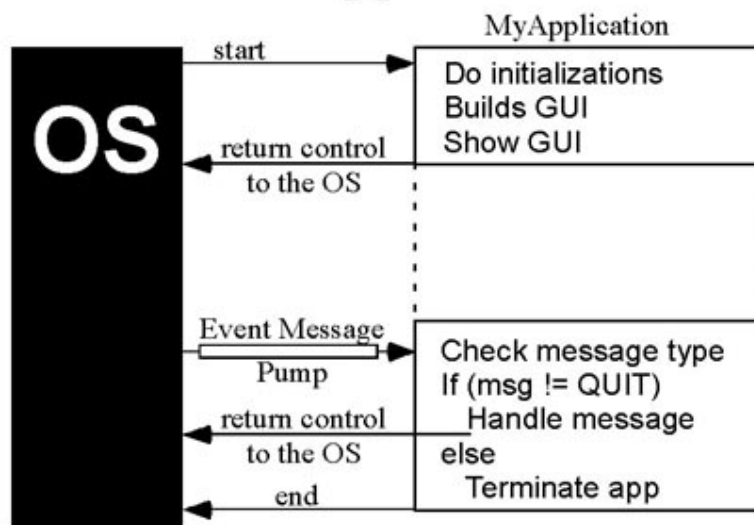
1. GUI Programming Basics

- **Different Approach:** Unlike console applications, GUI applications have a distinct execution paradigm.

Console Application



GUI Application



- **Two Main Parts:** GUI programs consist of
 - Initialization and setup
 - Event handling

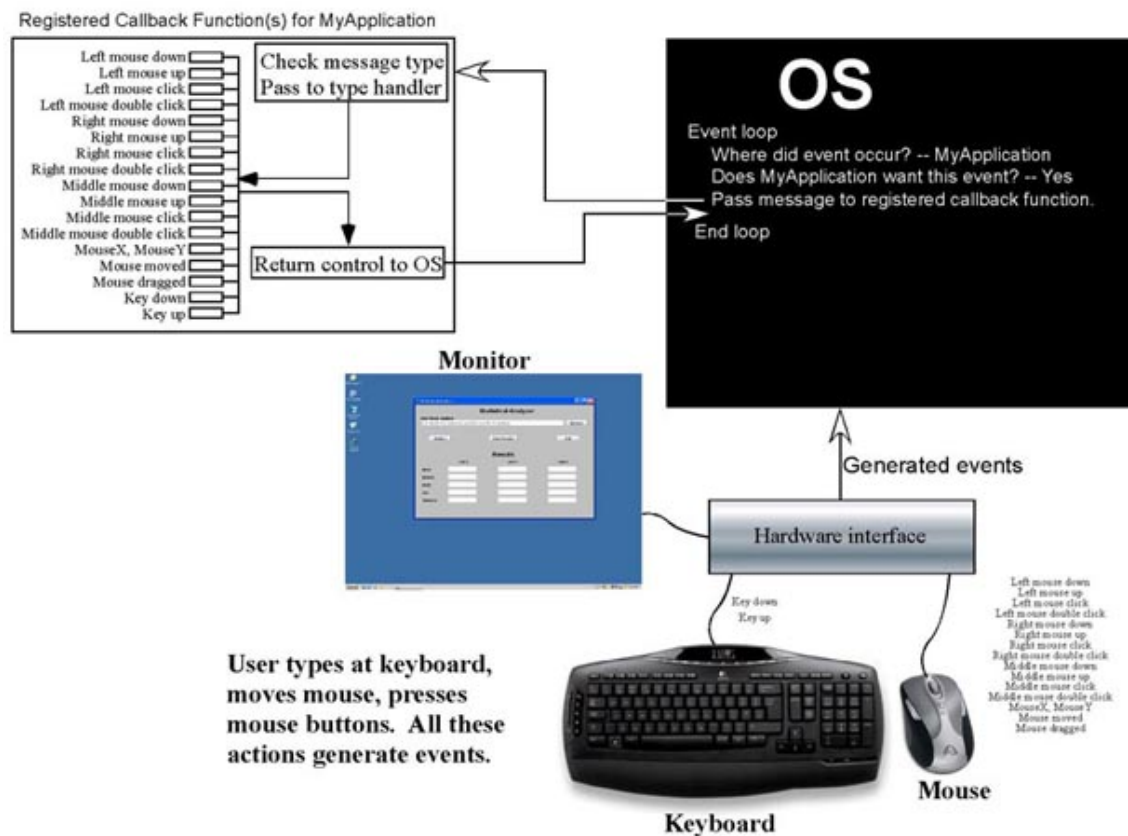
2. Initialization and Setup

- **Data Initialization:** Initialize data structures, instantiate classes, set variable values, etc.
- **Build GUI:** Place widgets (windows, dialog boxes, buttons, etc.) and define the events they handle.
- **Callback Functions:** Define and register callback functions with the operating system.

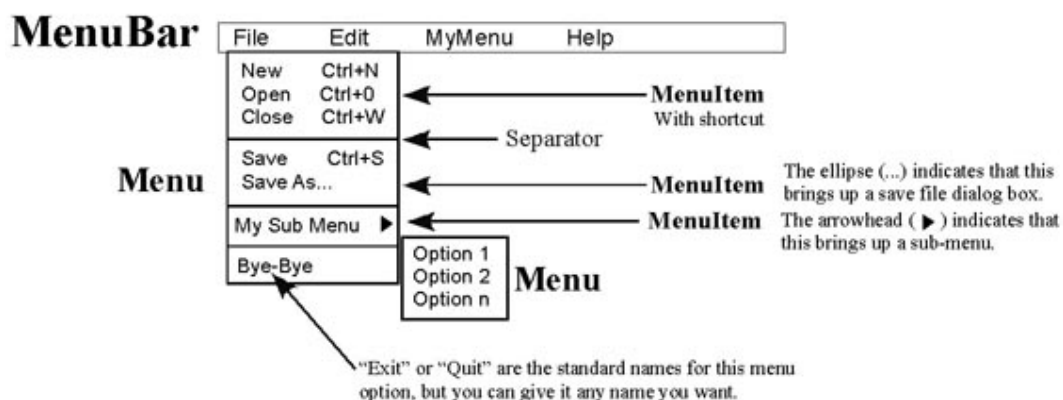
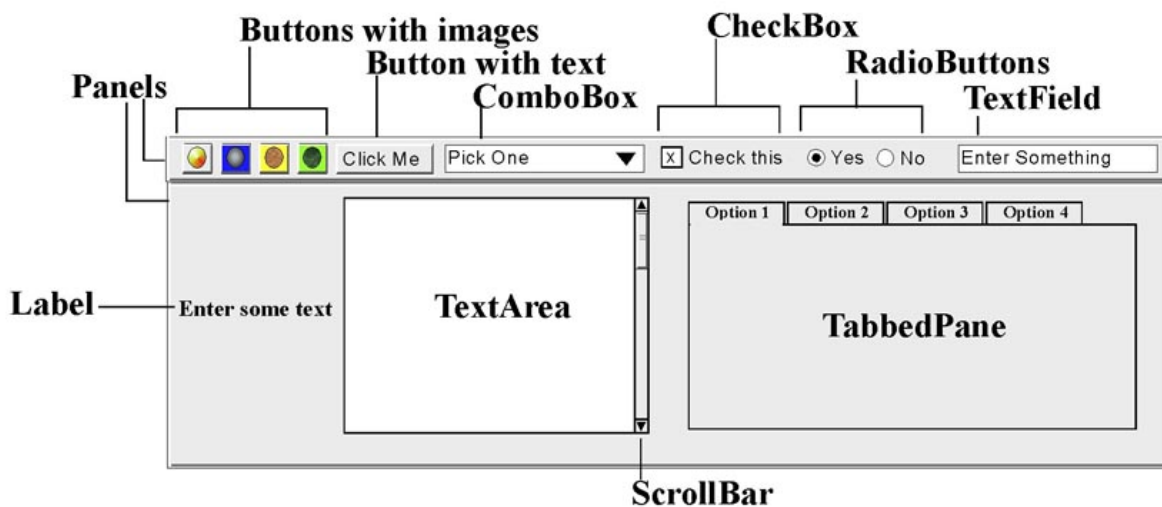
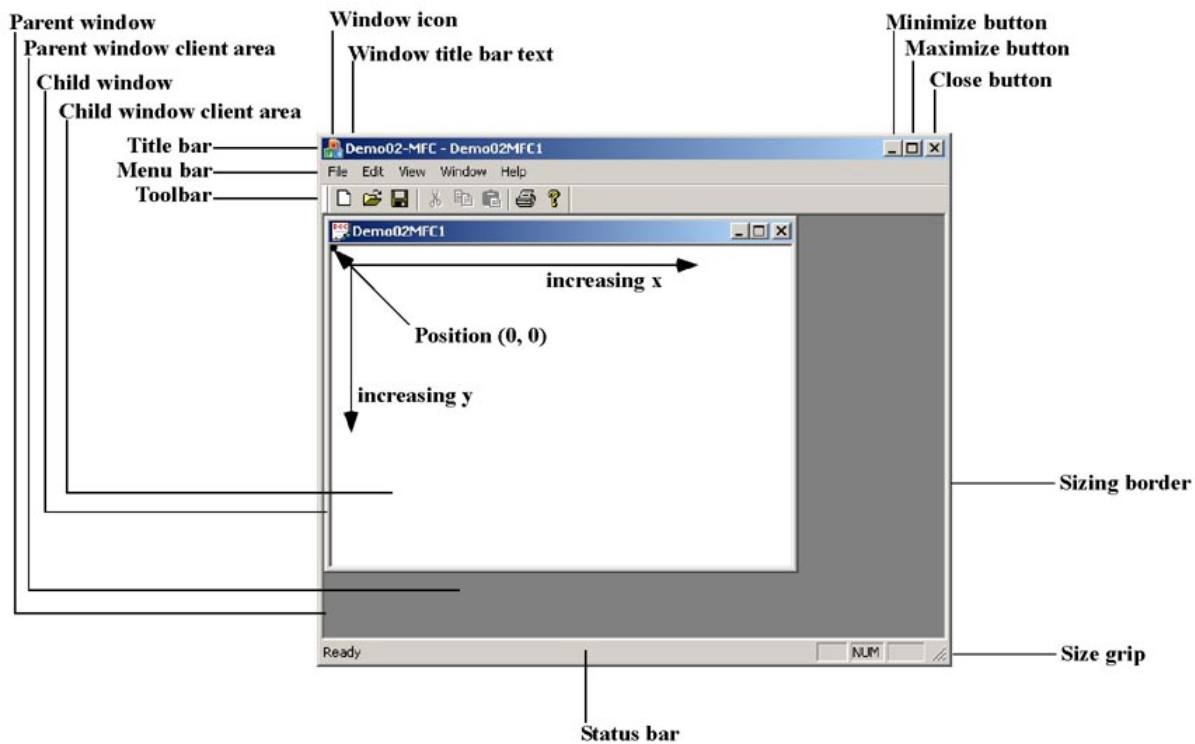
- **Make GUI Visible:** Make the application windows visible and return control to the OS.

3. Event Handling

- **Continuous Processing:** After exiting `main()`, the OS continues to process events.
- **Event Messages:** The OS packages events into "event message" structures and passes them to the application.
- **Event-Driven Programming:** Handle events generated by user interactions (mouse clicks, key presses, etc.).



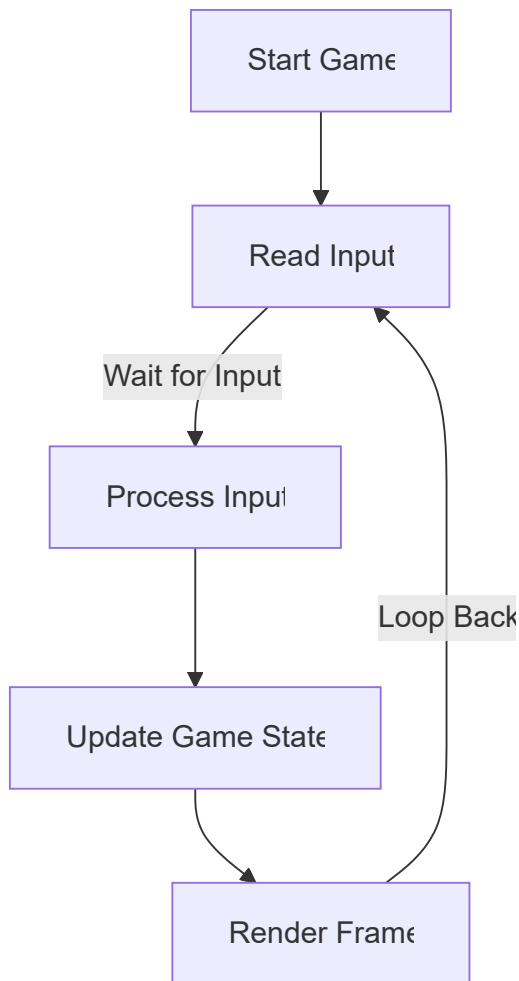
Parts of a GUI



GUI Programs vs Game Programs

Basic Game Loop

- **Continuous Processing:** Unlike event loops, game loops process input continuously and update the game state.
-
- **Three Main Steps:**
 - **process Input:** Handles user input.
 - **update:** Advances the game simulation.
 - **render:** Draws the game.



Timing and Synchronization

- **Frames Per Second (FPS):** The game loop cycles determine the game's FPS, influencing smoothness.
- **Consistent Speed:** Modern games must run at a consistent speed across different hardware, unlike older games tied to specific hardware.

Game Library

SFML Library

SFML and Linux

- **SFML** (Simple and Fast Multimedia Library) is a free, open-source, cross-platform library designed for multimedia application development. It provides an easy-to-use API for handling various multimedia components, making it a popular choice for game development and other graphical applications.

Features of SFML

- SFML is organized into several modules, each addressing specific aspects of multimedia programming:

1. System Module:

- Provides basic data types and utilities.
- Handles time, threads, and other low-level functionalities.

2. Window Module:

- Manages window creation, input handling, and OpenGL context settings.
- Provides an abstraction for keyboard, mouse, and joystick input.

3. Graphics Module:

- Offers 2D graphics rendering capabilities.
- Supports sprites, shapes, text, and textures.
- Includes basic transformations, such as scaling and rotation.

4. Audio Module:

- Enables sound and music playback.
- Supports audio streams and basic sound effects.
- Works with formats like WAV, OGG, and FLAC.

5. Network Module:

- Provides tools for network communication using TCP and UDP protocols.
- Handles sockets, packet serialization, and more.

Why Use SFML?

- **Ease of Use:** SFML has a straightforward API that makes it beginner-friendly.
- **Lightweight:** The library is optimized for performance and simplicity.
- **Cross-Platform:** Works on Windows, macOS, Linux, and other platforms.
- **Integration with C++:** Written in C++, SFML takes advantage of modern C++ features.
- **Extensibility:** Can integrate with other libraries, such as OpenGL for advanced rendering.

Typical Applications

- 2D games
- Interactive multimedia applications
- Prototyping and educational projects

Installing the SFML Library in Linux

If the version of SFML that you want to install is available in the official repository, then install it using your package manager. Use the following command in Debian based system like Ubuntu:

```
sudo apt-get install libsFML-dev
```

Example of a SFML library code

Assume that the following code is available in prog.cpp

```
#include <SFML/Graphics.hpp>

int main()
{
    sf::RenderWindow window(sf::VideoMode(200, 200), "SFML works!");
    sf::CircleShape shape(100.f);
    shape.setFillColor(sf::Color::Green);
    while (window.isOpen())
    {
        sf::Event event;
        while (window.pollEvent(event))
        {
            if (event.type == sf::Event::Closed)
                window.close();
        }
        window.clear();
        window.draw(shape);
        window.display();
    }
    return 0;
}
```

Compilation Process :

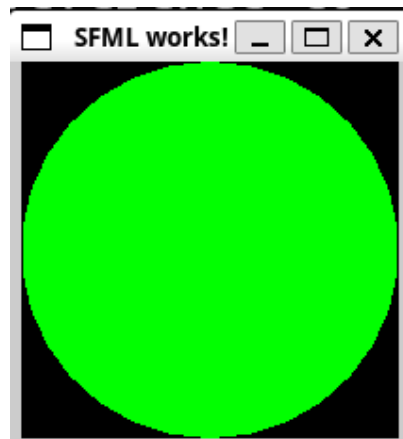
```
g++ prog.cpp -o my-app -lsfml-graphics -lsfml-window -lsfml-system
```

- `g++` is the GNU C++ compiler, which compiles the source code written in C++.
- `prog.cpp` is the source file containing the C++ code to compile.
- `-o my-app` specifies the name of the output executable file as `my-app`.
- `-lsfml-graphics` links the SFML Graphics module to your program. It provides support for rendering 2D graphics like sprites, shapes, and text.
- `-lsfml-window` links the SFML Window module, which handles window creation, OpenGL contexts, and input events such as keyboard and mouse interactions.
- `-lsfml-system` links the SFML System module. It provides utilities like time handling, threading, and other fundamental functions required by other SFML modules.

Running the executable file

```
./my-app
```

Output



Core Modules

The **Core of SFML** consists of five main modules:

1. System Module (`sf::System`)

- This is the lowest-level module and provides utilities such as:
 - `sf::Clock` → Measures elapsed time.
 - `sf::Time` → Represents time durations.
 - `sf::Sleep` → Puts the thread to sleep for a given duration.
 - `sf::Thread` and `sf::Mutex` → Multithreading support.
 - `sf::Vector2<T>` and `sf::Vector3<T>` → Mathematical vector utilities.

2. Window Module (`sf::Window`)

- Handles window creation and user input events.

- Features:
 - `sf::Window` → Creates and manages an application window.
 - `sf::Event` → Captures keyboard, mouse, and joystick inputs.
 - `sf::VideoMode` → Defines resolution, color depth, and refresh rate.
 - `sf::ContextSettings` → Configures OpenGL settings.
 - `sf::Joystick` → Manages game controller inputs.

3. Graphics Module (`sf::Graphics`)

- Provides 2D graphics rendering.
- Features:
 - `sf::RenderWindow` → Extends `sf::Window` with 2D rendering.
 - `sf::Texture` → Handles images.
 - `sf::Sprite` → Displays images efficiently.
 - `sf::Shape` , `sf::RectangleShape` , `sf::CircleShape` → Simple geometric shapes.
 - `sf::Font` and `sf::Text` → Loads fonts and renders text.
 - `sf::View` → Handles camera movement and zooming.

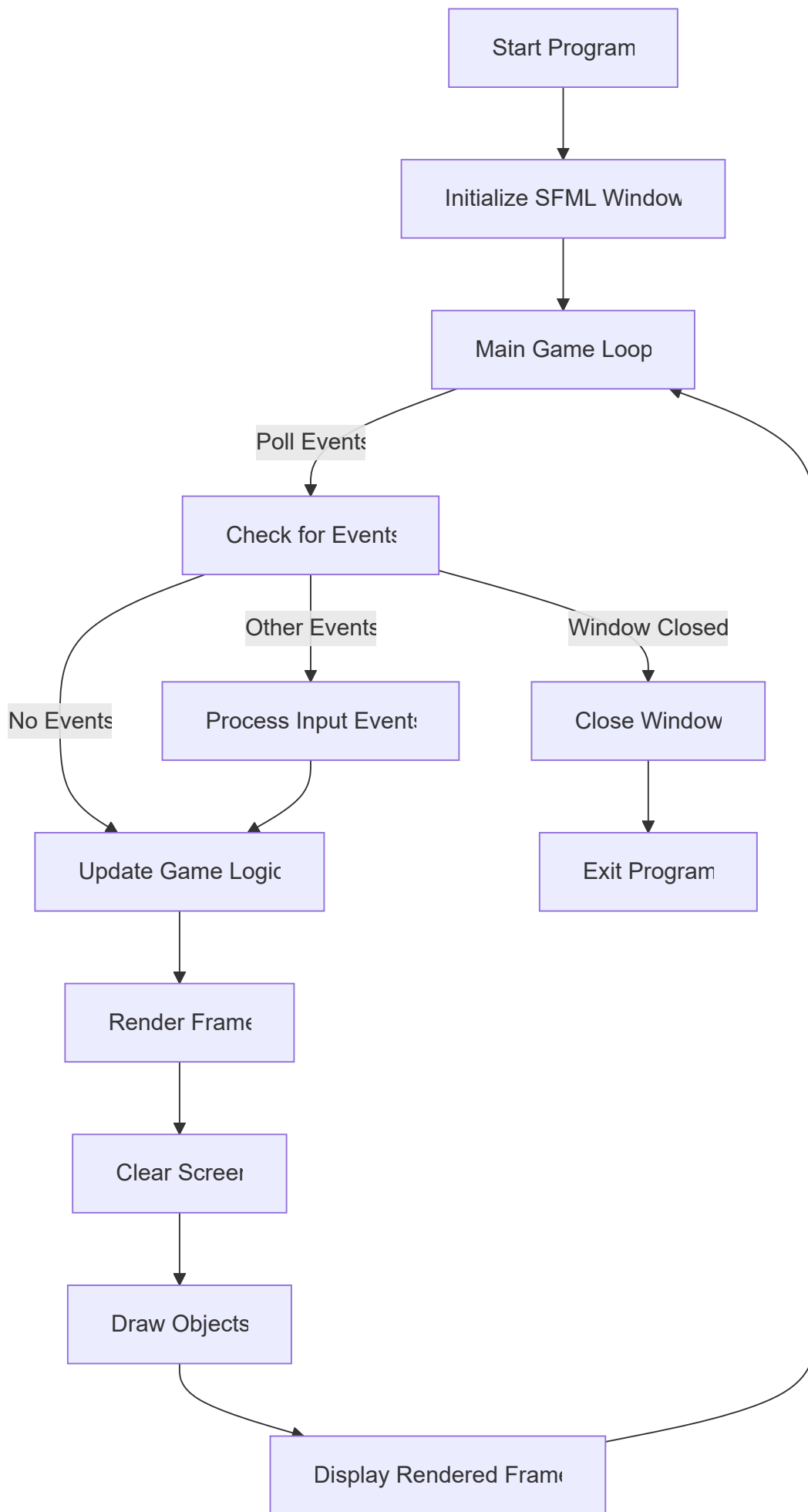
4. Audio Module (`sf::Audio`)

- Provides high-level audio playback.
- Features:
 - `sf::SoundBuffer` → Stores short sound samples.
 - `sf::Sound` → Plays short sounds from `sf::SoundBuffer` .
 - `sf::Music` → Streams large audio files.
 - `sf::Listener` → Simulates 3D audio.
 - `sf::SoundSource` → Base class for sound playback.

5. Network Module (`sf::Network`)

- Provides networking capabilities (TCP and UDP).
- Features:
 - `sf::IpAddress` → Manages IP addresses.
 - `sf::TcpSocket` , `sf::UdpSocket` → Handles network communication.
 - `sf::Packet` → Serializes data for transmission.
 - `sf::Http` and `sf::Ftp` → Allows HTTP and FTP requests.

Stages of SFML Game Program



Opening a window using SFML

```
// Include important libraries here
#include <SFML/Graphics.hpp>

// Make code easier to type with "using namespace"
using namespace sf;

// This is where our game starts from
int main() {

    // Create a video mode object
    VideoMode vm(1920, 1080);

    // Create and open a window for the game
    RenderWindow window(vm, "Timber!!!", Style::Fullscreen);

    while(1);
    return 0;
}
```

1. sf::VideoMode Class

The `sf::VideoMode` class in SFML represents the video mode of a window, i.e., its resolution (width & height) and bit depth (color depth in bits per pixel - BPP).

```
class sf::VideoMode
```

Common Constructors:

```
sf::VideoMode();
sf::VideoMode(unsigned int width, unsigned int height,
               unsigned int bitsPerPixel = 32);
```

Example:

```
// Create a 1920x1080 video mode with a default bit depth (32 BPP)
sf::VideoMode vm(1920, 1080);
```

- Here, 1920x1080 is the resolution.
- The bit depth defaults to **32 bits per pixel (BPP)** for full-color rendering.

Important Methods:

```
sf::VideoMode::getDesktopMode();
sf::VideoMode::getFullscreenModes();
```

- `getDesktopMode()` → Gets the current desktop resolution.
- `getFullscreenModes()` → Returns a list of supported fullscreen modes.

Example - Getting Desktop Resolution:

```
sf::VideoMode desktop = sf::VideoMode::getDesktopMode();
std::cout << "Desktop Resolution: " << desktop.width
           << "x" << desktop.height << std::endl;
```

2. `sf::RenderWindow` Class

The `sf::RenderWindow` class is a window that **can render graphics**. It extends `sf::Window` and adds rendering functionalities.

```
class sf::RenderWindow : public sf::Window
```

- `RenderWindow` is a class in SFML used to create a window for rendering graphics.
- The window is created using the `vm` (1920x1080 resolution) and given the title "Timber!!!".
- The **third parameter** (`Style::Fullscreen`) makes the window **full-screen**.
 - Other styles available in SFML:
 - `Style::Default` → Standard window with a title bar and close button.
 - `Style::Close` → A window that can only be closed.
 - `Style::Fullscreen` → Full-screen mode.

Main Game loop and Use Esc key to exit

Now we want our game to have a loop where we can design and update the scene as well as set the sequence to exit the full screen mode.

```
#include <SFML/Graphics.hpp>

// Make code easier to type with "using namespace"
using namespace sf;

int main()
{
    VideoMode vm(1920, 1080); // Create a video mode object

    // Create and open a window for the game
```

```

RenderWindow window(vm, "Timber!!!", Style::Fullscreen);

while (window.isOpen())
{
    if (Keyboard::isKeyPressed(Keyboard::Escape)) // Handel player
input
    {
        window.close();
    }

    /*
    Update the scene
    */

    /*
    Draw the scene
    */

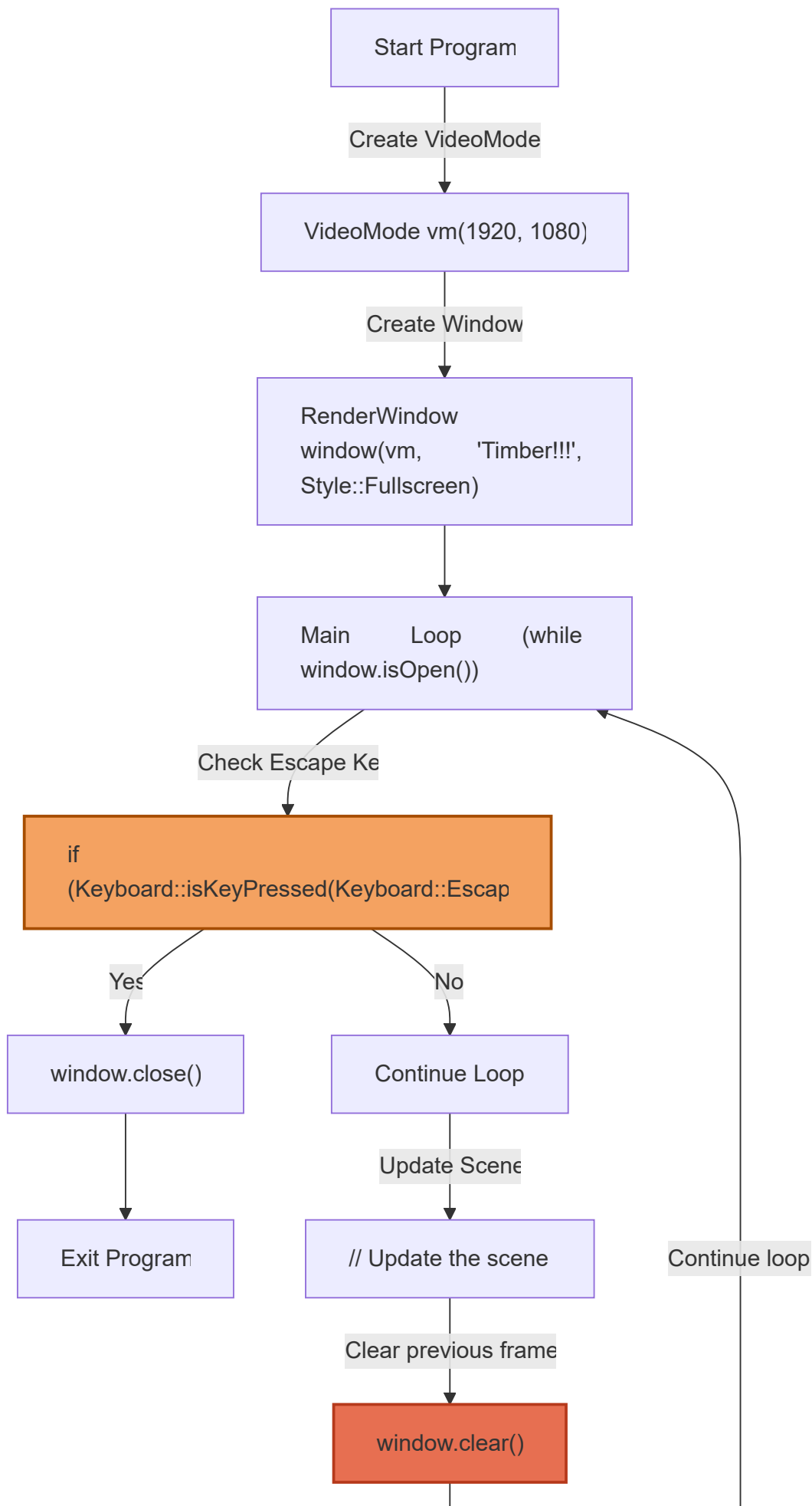
    // Clear everything from the last frame
    window.clear();

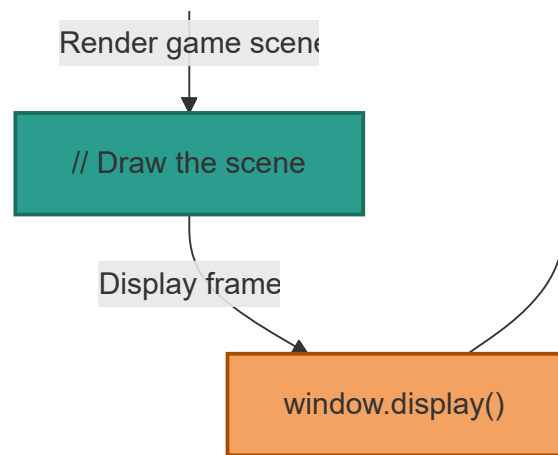
    // Draw our game scene here

    // Show everything we just drew
    window.display();
}
return 0;
}

```

- The function `Keyboard::isKeyPressed(Keyboard::Escape)` is a **static function** provided by SFML (Simple and Fast Multimedia Library) to check whether a specific key is currently being pressed.
- `Keyboard` : This is a class in SFML that provides access to keyboard inputs.
- `isKeyPressed()` : A **static function** that checks whether a key is currently being pressed.
- `Keyboard::Escape` : This refers to the **Escape key** on the keyboard.





Sprites in a game

A **sprite** is a 2D image or animation used in a game to represent characters, objects, or effects. It is a key element in 2D game development and is typically rendered on the screen as a graphical object.

Key Features of a Sprite:

- **2D Representation** – Unlike 3D models, sprites are flat images displayed on a 2D plane.
- **Moveable & Interactive** – Sprites can be animated, moved, or manipulated in the game.
- **Layered Rendering** – Sprites are drawn on different layers (foreground, midground, background) to create depth.
- **Transparent Background** – Most sprites use PNG images with transparency to blend into the game scene.

Types of Sprites:

1. **Static Sprites** – A single image that does not change (e.g., a tree or rock).
2. **Animated Sprites** – A sequence of images (frames) that create an animation (e.g., walking, jumping).
3. **Sprite Sheets** – A collection of images arranged in a grid to optimize animation and performance.
4. **Tiled Sprites** – Small repeating textures used to create large environments like floors and walls.

How Sprites Work in Games:

- **Rendering:** Sprites are drawn by the game engine onto the screen.
- **Collision Detection:** They interact with other objects based on bounding boxes or pixel detection.
- **Transformation:** They can be moved, rotated, scaled, or flipped based on game logic.

- **Animation:** Frame-by-frame changes give the illusion of movement (e.g., walking cycle).

Where Sprites are Used:

- **2D Games:** Platformers, RPGs, puzzle games (e.g., *Super Mario Bros.*, *Celeste*).
- **UI Elements:** Buttons, icons, menus in both 2D and 3D games.
- **Special Effects:** Fire, smoke, explosions, magic spells.

Types of sprites

1. Character Sprites

- **Player Character (PC)** – The main character controlled by the player.
- **Non-Player Characters (NPCs)** – Characters that are AI-controlled, such as villagers, shopkeepers, or quest givers.
- **Enemies** – Hostile characters that challenge the player, including monsters, soldiers, or bosses.

2. Environment Sprites

- **Background Sprites** – Static or layered images that create the backdrop of the game (e.g., sky, mountains, buildings).
- **Foreground Sprites** – Elements that appear in front of characters, such as trees, fences, or rocks.
- **Platforms and Terrain** – Ground, bridges, floating platforms, and surfaces that characters interact with.

3. Interactive Object Sprites

- **Collectibles** – Coins, gems, power-ups, or keys that the player can collect.
- **Weapons & Tools** – Guns, swords, shields, or tools used by the player or NPCs.
- **Doors & Switches** – Objects that trigger events when interacted with.

4. Animated Effect Sprites

- **Particle Effects** – Fire, smoke, rain, explosions, or magic effects.
- **Hit Effects** – Flashes, damage indicators, or destruction effects.
- **UI Indicators** – Floating damage numbers, health bars, or quest markers.

5. UI and HUD Sprites

- **Health & Mana Bars** – Displays the status of the player's health and resources.
- **Icons & Buttons** – Inventory icons, ability icons, or in-game buttons.
- **Text and Dialog Boxes** – UI elements that display conversations or information.

6. Parallax Sprites

- **Multiple Background Layers** – Used to create a depth effect where background layers move at different speeds.

How a Game Works: Rendering and Sprite Interaction

A game operates through a continuous **game loop**, which involves:

1. **Processing Input** – Reads player actions (keyboard, mouse, controller).
2. **Updating Game State** – Moves objects, detects collisions, and applies physics.
3. **Rendering Graphics** – Draws game elements on the screen.
4. **Repeating** – Runs at a fixed frame rate (e.g., 60 FPS).

Rendering Process

- **Game Window Setup** – Initializes resolution, aspect ratio, and rendering context.
- **Loading Sprites** – Stores textures in GPU memory for efficient access.
- **Drawing Scene** – Clears the screen and draws objects in layers (background, sprites, UI).
- **Frame Buffer & VSync** – Ensures smooth rendering by managing refresh rates.

Sprite Interaction

- **Movement & Physics** – Sprites respond to user input, velocity, and gravity.
- **Collision Detection** – Detects interactions using bounding boxes, circles, or pixel-perfect methods.
- **Handling Collisions** – Adjusts positions, triggers effects, and executes game logic.

Game Logic & Optimization

- **Game Events** – Manages objectives, health, and AI behavior.
- **Rendering Optimizations** – Uses techniques like batch rendering, Level of Detail (LOD), and culling.
- **State Management** – Handles pause, save/load, and multiplayer synchronization.

Timberman Game

Timberman Game - Objective and Description for Design

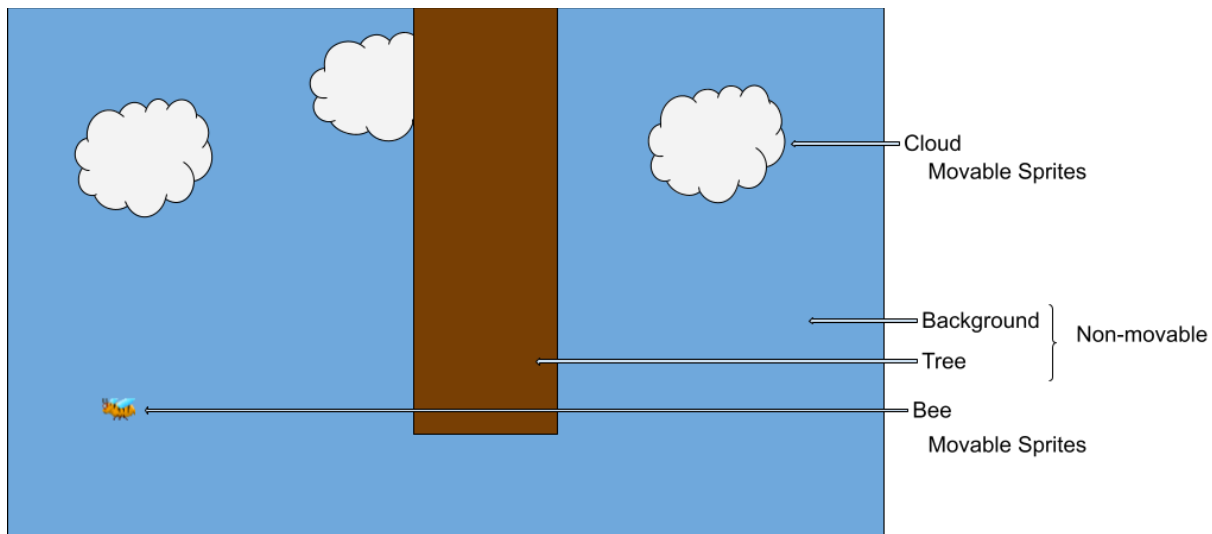
Objective:

The primary goal of **Timberman** is to chop down a tree as fast as possible while avoiding branches. The player must continuously chop to keep the timer from running out. The game ends if the player gets hit by a branch or the timer depletes.

Game Description:

- **Gameplay:** The player controls a lumberjack who stands beside a tall tree. They must chop from either the left or right side while avoiding oncoming branches. The game is fast-paced, requiring quick reflexes and decision-making.
- **Controls:** Typically, the player taps or presses keys to chop the tree on either side.
- **Mechanics:**
 - The tree falls continuously as the player chops.
 - Each chop scores points and slightly refills the timer.
 - Hitting a branch results in an instant game over.
 - The longer the player chops, the faster the tree falls, increasing difficulty.
 - Some versions introduce power-ups or new characters with special abilities.
- **Visual Design:**
 - Simple, pixel-art or cartoonish aesthetic.
 - Background changes with progression.
 - Characters can be customizable or unlockable.
 - Smooth animation for chopping, tree falling, and branch movement.

Consider Environment Sprites



```
//Adding Background
#include<SFML/Graphics.hpp>
using namespace sf;

int main()
{
    VideoMode vm(1920,1080);

    RenderWindow window(vm, "Timber !!!");

    // Create a texture to hold a graphic on the GPU
    Texture textureBackground;

    // Load a graphic into the texture
    textureBackground.loadFromFile("background.png");

    // Create a sprite
    Sprite spriteBackground;

    // Attach the texture to the sprite
    spriteBackground.setTexture(textureBackground);

    // Set the spriteBackground to cover the screen
    spriteBackground.setPosition(0, 0);

    while(window.isOpen())
    {
        if (Keyboard::isKeyPressed(Keyboard::Escape)) // Handel player
input
        {
            window.close();
        }
    }
}
```

```

        window.clear();

        //Drawing the background sprite
        window.draw(spriteBackground);

        window.display();
    }
}

```

sf::RenderWindow Class

Purpose:

Represents a window where graphical content is displayed.

Member Functions Used:

- **Constructor:** `RenderWindow(VideoMode mode, const std::string& title)`
 - Creates a window with the specified resolution and title.
- `isOpen()`
 - Returns `true` if the window is open, else `false`.
- `close()`
 - Closes the window.
- `clear()`
 - Clears the window and prepares it for new rendering.
- `draw(const Drawable& object)`
 - Draws a drawable object (like `Sprite`) to the window.
- `display()`
 - Displays everything drawn onto the window.

sf::Texture Class

Purpose:

Holds an image to be used for rendering.

Member Function Used:

- `loadFromFile(const std::string& filename)`
 - Loads an image file into the texture.

Usage in Code:

```

Texture textureBackground;
textureBackground.loadFromFile("background.png");

```

- Loads the image "background.png" into the textureBackground object.

sf::Sprite Class

Purpose:

Represents a graphical object that can be drawn onto the window.

Member Functions Used:

- `setTexture(const Texture& texture)`
 - Sets a texture for the sprite.
- `setPosition(float x, float y)`
 - Sets the sprite's position in the window.

Understanding the Game coordinates

In 2D games, everything happens on a **flat surface**, like a graph with an X and Y axis.

Coordinate System Basics

- **X-axis:** Horizontal (left ↔ right)
- **Y-axis:** Vertical (up ↑ down)

A position is represented as:

(x, y)

For example:

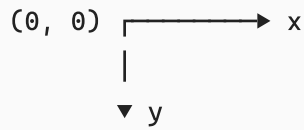
- (0, 0) is the **origin** — usually the top-left or bottom-left corner depending on the engine.
- (100, 200) means:
 - 100 units **right**
 - 200 units **down** or **up**, depending on the coordinate system

Top-Left vs Bottom-Left Origin

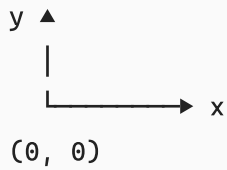
Coordinate System	Origin	Y increases...
Screen-based	Top-left corner	Going down
Math-based	Bottom-left corner	Going up

Visual Example:

Screen-based (Image coordinate system)



Math-based (Coordinate system)



What Coordinates Are Used For

- Player or object position
- Movement (updating x , y)
- Collision detection
- Camera tracking
- Mouse/touch input

```
//Add a movable sprite and add movement to it
#include<SFML/Graphics.hpp>
using namespace sf;

int main()
{
    VideoMode vm(1920,1080);

    RenderWindow window(vm, "Timber !!!");

    Texture textureCloud;

    textureCloud.loadFromFile("cloud.png");

    Sprite spriteCloud;

    spriteCloud.setTexture(textureCloud);

    spriteCloud.setPosition(0, 200);

    while(window.isOpen())
    {
        if (Keyboard::isKeyPressed(Keyboard::Escape)) // Handel player
input
    {
```

```

        window.close();
    }

    //Adding movement logic
    spriteCloud.setPosition(spriteCloud.getPosition().x+20,200);

    window.clear();

    //Drawing the cloud sprite
    window.draw(spriteCloud);

    window.display();
}
}

```

NB: You will see the a cloud moving from left to right. We achieve that by changing the x coordinate of cloud sprite in each frame.

But we don't want the cloud to move out of the window. So as soon as the cloud reaches the right border we want the cloud to restart from left again.

```

//Add a movable sprite and add movement to it
#include<SFML/Graphics.hpp>
using namespace sf;

int main()
{
    VideoMode vm(1920,1080);

    RenderWindow window(vm, "Timber !!!");

    Texture textureCloud;

    textureCloud.loadFromFile("cloud.png");

    Sprite spriteCloud;

    spriteCloud.setTexture(textureCloud);

    spriteCloud.setPosition(0, 200);

    while(window.isOpen())
    {
        if (Keyboard::isKeyPressed(Keyboard::Escape)) // Handel player
input
        {
            window.close();

```

```

    }

    //Adding movement logic
    if(spriteCloud.getPosition().x > 1920)
        spriteCloud.setPosition(0, 200);
    else
        spriteCloud.setPosition(spriteCloud.getPosition().x+20,200);

    window.clear();

    //Drawing the cloud sprite
    window.draw(spriteCloud);

    window.display();
}
}

```

Note

- The movement depends on the frame which is highly unreliable for real time use as frame rendering may not happen with equal time interval.
- Hence, we may want to add the movement to the time elapsed to make sure movement of the sprite according to time interval.

```

//Add a movable sprite and add movement to it

#include<SFML/Graphics.hpp>
using namespace sf;

int main()
{
    VideoMode vm(1920,1080);

    RenderWindow window(vm, "Timber !!!");

    Texture textureCloud;

    textureCloud.loadFromFile("cloud.png");

    Sprite spriteCloud;

    spriteCloud.setTexture(textureCloud);

    spriteCloud.setPosition(0, 200);

    float cloudSpeed = 15; // With how much time it should cover the

```

```

window width
    float cloudSpeedPerSec = 1920/cloudSpeed;

    Time dt; // Object to manipulate the time
    Clock ct; // Object to have a Clock Counter

    while(window.isOpen())
    {
        if (Keyboard::isKeyPressed(Keyboard::Escape))
        {
            window.close();
        }
        dt = ct.restart();

        //Adding movement logic
        if(spriteCloud.getPosition().x > 1920)
            spriteCloud.setPosition(0, 200);
        else
            spriteCloud.setPosition(spriteCloud.getPosition().x+
                                    (dt.asSeconds()*cloudSpeedPerSec),200);

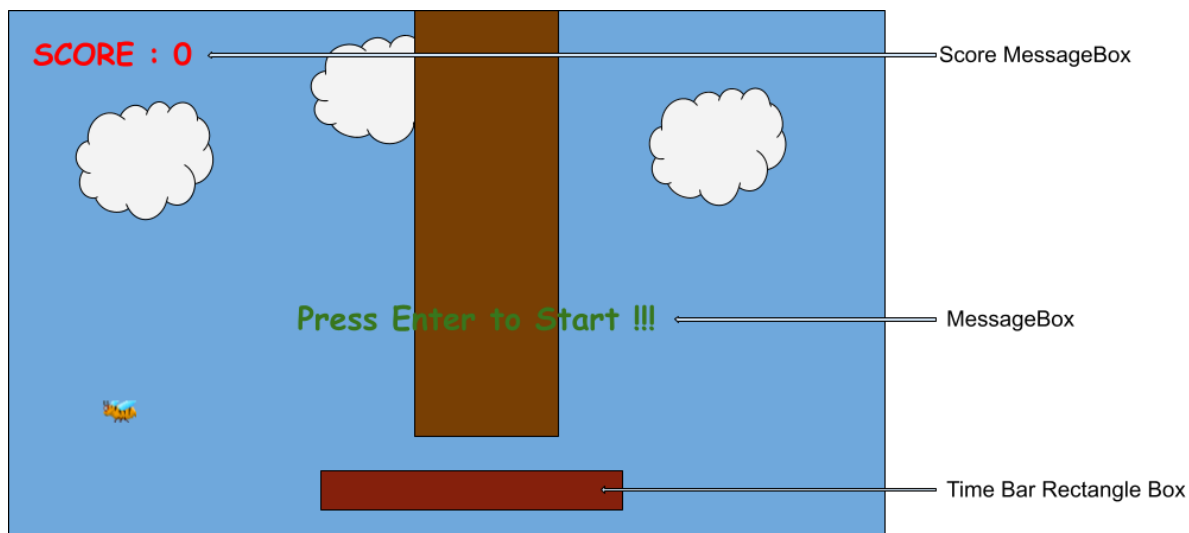
        window.clear();

        //Drawing the cloud sprite
        window.draw(spriteCloud);

        window.display();
    }
}

```

Consider UI and HUD Sprites



What is a HUD?

HUD (Heads-Up Display) refers to on-screen elements that show information like:

- Score
- Health bars
- Ammo count
- Timer
- Minimap

Adding a Score Display (Text UI Element)

To show a score in SFML:

- Use `sf::Font` to load a font.
- Use `sf::Text` to render the score.

Example

```
#include <SFML/Graphics.hpp>
#include <string>

int main() {
    sf::RenderWindow window(sf::VideoMode(800, 600), "Timber !!!");

    // Load font
    sf::Font font;
    font.loadFromFile("KOMIKAP_.ttf");

    // Score Text Setup
    int score = 0;
    sf::Text scoreText;
    scoreText.setFont(font);
    scoreText.setCharacterSize(30);
    scoreText.setFillColor(sf::Color::White);
    scoreText.setPosition(10, 10);
    scoreText.setString("Score: 0");

    // Main loop
    while (window.isOpen()) {
        sf::Event event;
        while (window.pollEvent(event)) {
            if (event.type == sf::Event::Closed)
                window.close();
        }

        // Update score
        score += 1;
        scoreText.setString("Score: " + std::to_string(score));
    }
}
```

```

        // Render
        window.clear();
        window.draw(scoreText);
        window.display();
    }

    return 0;
}

```

Time-Bar HUD

Objective

- Display a time bar (like a health/progress bar) at the **bottom** of the screen.
- The bar **shrinks** over a fixed duration (e.g., 10 seconds).
- After time runs out, the bar is gone (empty).

```

#include <SFML/Graphics.hpp>
using namespace sf;

int main() {
    sf::RenderWindow window(sf::VideoMode(800, 600), "SFML Time Bar");

    // Set total countdown time (in seconds)
    float totalTime = 10.0f;
    float elapsedTime = 0.0f;

    // Initial size of the time bar
    float barWidth = 700.0f;
    float barHeight = 20.0f;

    // Time bar setup
    RectangleShape timeBar;
    timeBar.setSize(sf::Vector2f(barWidth, barHeight));
    timeBar.setFillColor(sf::Color::Red);
    timeBar.setPosition(50, 570); // bottom with margin

    // Clock to track elapsed time
    Clock ct;
    Time dt;

    while (window.isOpen()) {
        dt = ct.restart();
        sf::Event event;
        while (window.pollEvent(event)) {
            if (event.type == sf::Event::Closed)
                window.close();
        }
    }
}

```

```

// Update elapsed time
elapsedTime += dt.asSeconds();

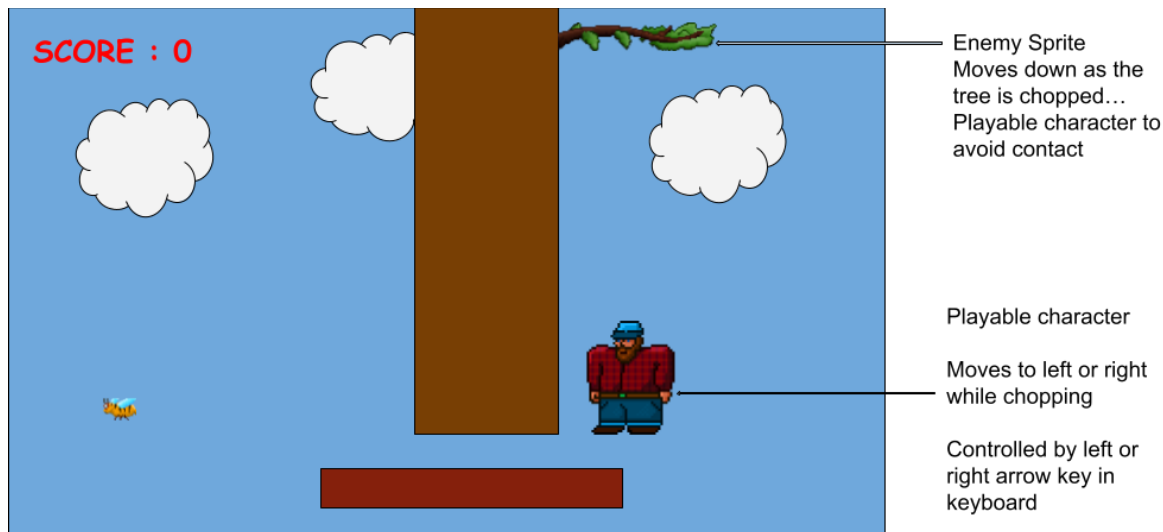
// Calculate the remaining ratio and update the time bar width
float remainingRatio = ((totalTime - elapsedTime) / totalTime);
if(remainingRatio > 0.0f)
    timeBar.setSize(sf::Vector2f(barWidth * remainingRatio,
barHeight));
else
    timeBar.setSize(sf::Vector2f(0, barHeight));

// Render everything
window.clear();
window.draw(timeBar);
window.display();
}

return 0;
}

```

Consider Character Sprites



```

// TimberMan Game with SFML
// Compile using: g++ TimberMan.cpp -o TimberMan -lsfml-graphics -lsfml-
window -lsfml-system -lsfml-audio

#include <SFML/Graphics.hpp>
#include <SFML/Audio.hpp>
#include <sstream>
#include <cmath>
using namespace sf;

// Updates cloud movement from left to right and resets when off screen

```

```

void cloudUpdate(Sprite &spriteCloud, Time &dt, float
&cloudPixelPerSecond) {
    float x = spriteCloud.getPosition().x;
    float y = spriteCloud.getPosition().y;
    if (x > 1980) {
        spriteCloud.setPosition(-200, y); // reset cloud to left
    } else {
        spriteCloud.setPosition(x + (dt.asSeconds() *
cloudPixelPerSecond), y); // move right
    }
}

// Updates bee movement with sine wave and resets position after off-
screen
void beeUpdate(Sprite &spriteBee, Time &dt, float &beePixelPerSecond) {
    static float timex = 0;
    static float y_dir = 820;
    timex += dt.asSeconds();

    float x = spriteBee.getPosition().x;
    float y = spriteBee.getPosition().y;

    if (x < 0) {
        y_dir = 820 + rand() % 200 - 100; // randomize vertical position
        spriteBee.setPosition(2000, y); // reset to right
    } else {
        y = y_dir + 50 * sin(timex); // vertical sine motion
        spriteBee.setPosition(x - (dt.asSeconds() * beePixelPerSecond),
y); // move left
    }
}

// Updates the on-screen score text
void scoreUpdate(Text &scoreText, int &score_val) {
    std::stringstream ss;
    ss << "Score = " << score_val;
    scoreText.setString(ss.str());
}

// Updates the time bar; handles time-out scenario
void timebarUpdate(RectangleShape &timebar, float &timeRemaining, bool
&pause, bool &timeOut,
                    Text &messageText, float &timebarHeight, float
&timebarWidthPerSecond, Time &dt, bool logActive) {
    if (!logActive) {
        timeRemaining -= dt.asSeconds();
        if (timeRemaining < 0) {
            pause = true;
            timeOut = true;
            messageText.setString("TimeOut!!!");

```



```

        timebar.setSize(Vector2f(0, timebarHeight)); // hide time bar
    } else {
        if (timeRemaining > 6.0f)
            timeRemaining = 6.0f;
        float newBarWidth = timeRemaining * timebarWidthPerSecond;
        timebar.setSize(Vector2f(newBarWidth, timebarHeight));
    }
}

// Shifts branches downward and spawns new one at the top
void branchUpdate(int &numBranches, Sprite *spriteBranch, float
&chopHeight) {
    for (int i = numBranches - 1; i > 0; i--) {
        float x = spriteBranch[i - 1].getPosition().x;
        float y = spriteBranch[i - 1].getPosition().y;
        if (y > 900) y = -200;
        else y += chopHeight;
        spriteBranch[i].setPosition(x, y);
        spriteBranch[i].setRotation(spriteBranch[i - 1].getRotation());
    }

    // Randomly place a new branch or leave empty
    int r = rand() % 3;
    float x, rotate = 0;
    if (r == 0) { x = 590; rotate = 180; } // Left
    else if (r == 1) { x = 1330; rotate = 0; } // Right
    else { x = 3000; } // No branch
    spriteBranch[0].setPosition(x, -200);
    spriteBranch[0].setRotation(rotate);
}

int main() {
    VideoMode vm(800, 600);
    RenderWindow window(vm, "Timber Man ");

    View view(FloatRect(0, 0, 1980, 1020));
    window.setView(view);

    // Load and setup background
    Sprite spriteBackground;
    Texture textureBackground;
    textureBackground.loadFromFile("background.png");
    spriteBackground.setTexture(textureBackground);

    // Load and set cloud sprites
    Sprite spriteCloud, spriteCloud2, spriteCloud3;
    Texture textureCloud;
    textureCloud.loadFromFile("cloud.png");
    spriteCloud.setTexture(textureCloud);

```

```

spriteCloud.setPosition(0, 50);
spriteCloud.setScale(0.3, 0.3);
spriteCloud2.setTexture(textureCloud);
spriteCloud2.setPosition(0, 80);
spriteCloud2.setScale(0.6, 0.6);
spriteCloud3.setTexture(textureCloud);
spriteCloud3.setPosition(0, 120);

float cloudPixelPerSecond = 1980 / 20.0f;
float cloud2PixelPerSecond = 1980 / 15.0f;
float cloud3PixelPerSecond = 1980 / 10.0f;

// Bee setup
Sprite spriteBee;
Texture textureBee;
textureBee.loadFromFile("bee.png");
spriteBee.setTexture(textureBee);
spriteBee.setPosition(1980, 820);
float beePixelPerSecond = 1980 / 15.0f;

// Tree
Sprite spriteTree;
Texture textureTree;
textureTree.loadFromFile("tree.png");
spriteTree.setTexture(textureTree);
spriteTree.setPosition(810, 0);

// Score display
int score_val = 0;
Text scoreText;
Font font;
font.loadFromFile("KOMIKAP_.ttf");
scoreText.setFont(font);
scoreText.setString("Score = 0");
scoreText.setPosition(20, 20);
scoreText.setCharacterSize(100);
scoreText.setFill(Color::Yellow);

// Message text (centered)
Text messageText;
messageText.setFont(font);
messageText.setCharacterSize(80);
messageText.setFill(Color::White);
messageText.setString("Press Enter to start the game !!!!");
messageText.setPosition(1980 / 2.0, 1020 / 2.0f);
FloatRect details = messageText.getLocalBounds();
messageText.setOrigin(details.left + details.width / 2.0f,
details.top + details.height / 2.0f);

// Time bar setup

```

```

RectangleShape timebar;
float timebarHeight = 80;
float tiembarWidth = 400;
timebar.setSize(Vector2f(tiembarWidth, timebarHeight));
timebar.setFill(Color::Red);
timebar.setPosition((1920 / 2.0f) - 200, 1020 - 140);
float timeRemaining = 6.0f;
float timebarWidthPerSecond = tiembarWidth / timeRemaining;

// Branches
int numBranches = 6;
Sprite spriteBranch[numBranches];
Texture textureBranch;
textureBranch.loadFromFile("branch.png");
for (int i = 0; i < numBranches; i++) {
    spriteBranch[i].setTexture(textureBranch);
    spriteBranch[i].setPosition(590, -200);
    spriteBranch[i].setOrigin(220, 40);
    spriteBranch[i].setRotation(180);
}

float chopHeight = 200;

// Player setup
Sprite spritePlayer;
Texture texturePlayer;
texturePlayer.loadFromFile("player.png");
spritePlayer.setTexture(texturePlayer);
spritePlayer.setPosition(1130, 708);

// Log setup
Sprite spriteLog;
Texture textureLog;
textureLog.loadFromFile("log.png");
spriteLog.setTexture(textureLog);
spriteLog.setPosition(810, 780);
float speedLog = 0.3;
float logPixelPexSecX = 990 / speedLog;
float logPixelPerSecY = 200 / speedLog;
int logDir = 0;

// Axe
Sprite spriteAxe;
Texture textureAxe;
textureAxe.loadFromFile("axe.png");
spriteAxe.setTexture(textureAxe);
spriteAxe.setPosition(2300, 800);

// Sound effect
Sound chop;

```

```

SoundBuffer chopSound;
chopSound.loadFromFile("chop.wav");
chop.setBuffer(chopSound);

// Rip sprite (death)
Sprite spriteRip;
Texture textureRip;
textureRip.loadFromFile("rip.png");
spriteRip.setTexture(textureRip);
spriteRip.setPosition(2300, 750);

// Game state flags
bool logActive = false, pause = true, timeOut = false, acceptInput =
true, gamestart = false, gameOver = false;
Clock ct;
Time dt;

// Main game loop
while (window.isOpen()) {
    dt = ct.restart();

    Event ev;
    while (window.pollEvent(ev)) {
        if (ev.type == Event::Closed) window.close();
        if (ev.type == Event::KeyReleased && !acceptInput) {
            spriteAxe.setPosition(2300, 800);
            acceptInput = true;
        }
    }

    // Input checks and gameplay logic
    if (acceptInput) {
        // Start or restart the game
        if (Keyboard::isKeyPressed(Keyboard::Enter)) {
            if (timeOut || gameOver) {
                pause = false;
                timeRemaining = 6.0f;
                score_val = 0;
                messageText.setString("");
                timeOut = false;
                gameOver = false;

                // Reset all branches and positions
                for (int i = 0; i < numBranches; i++)
                    spriteBranch[i].setPosition(3000, -200);

                spritePlayer.setPosition(1130, 708);
                spriteRip.setPosition(2300, 750);
                spriteLog.setPosition(810, 780);
                logActive = false;
            }
        }
    }
}

```

```

        spritePlayer.setScale(1, 1);
    } else {
        gamestart = true;
        pause = false;
        messageText.setString("");
    }
    acceptInput = false;
}

// Pause the game
if (Keyboard::isKeyPressed(Keyboard::Space) && gamestart) {
    pause = !pause;
    messageText.setString(pause ? "Game is Paused !!!" : "");
    acceptInput = false;
}

// Chop left
if (Keyboard::isKeyPressed(Keyboard::Left) && !pause) {
    score_val++;
    timeRemaining += (2.0f / score_val) + 0.2f;
    branchUpdate(numBranches, spriteBranch, chopHeight);
    spritePlayer.setPosition(790, 708);
    spriteAxe.setPosition(740, 800);
    spritePlayer.setScale(-1, 1);
    logActive = true;
    logDir = 1;
    chop.play();
    acceptInput = false;
}

// Chop right
if (Keyboard::isKeyPressed(Keyboard::Right) && !pause) {
    score_val++;
    timeRemaining += (2.0f / score_val) + 0.2f;
    branchUpdate(numBranches, spriteBranch, chopHeight);
    spritePlayer.setPosition(1130, 708);
    spritePlayer.setScale(1, 1);
    spriteAxe.setPosition(1040, 800);
    logActive = true;
    logDir = -1;
    chop.play();
    acceptInput = false;
}

}

// Move the log if it's active
if (logActive && !pause) {
    acceptInput = false;
    float x = spriteLog.getPosition().x;
    float y = spriteLog.getPosition().y;

```

```

        if (x < -100 || x > 2000) {
            // Log goes off screen - reset
            x = 810;
            y = 780;
            logActive = false;
            acceptInput = true;
        } else {
            // Move the log diagonally
            y -= logPixelPerSecY * dt.asSeconds();
            x += logDir * logPixelPerSecX * dt.asSeconds();
        }
        spriteLog.setPosition(x, y);
    }

    // Update game objects if not paused
    if (!pause) {
        cloudUpdate(spriteCloud, dt, cloudPixelPerSecond);
        cloudUpdate(spriteCloud2, dt, cloud2PixelPerSecond);
        cloudUpdate(spriteCloud3, dt, cloud3PixelPerSecond);
        beeUpdate(spriteBee, dt, beePixelPerSecond);
        scoreUpdate(scoreText, score_val);
        timebarUpdate(timebar, timeRemaining, pause, timeOut,
messageText,
                                timebarHeight, timebarWidthPerSecond, dt,
logActive);
    }

    // Game over condition (player hit by branch)
    if
(spritePlayer.getGlobalBounds().intersects(spriteBranch[numBranches -
1].getGlobalBounds())) {
        gameOver = true;
        pause = true;
        score_val = 0;
        messageText.setString("GAME OVER !!!!!");
        timeRemaining = 0;
        timebar.setSize(Vector2f(0, timebarHeight));
        float x = spritePlayer.getPosition().x;
        spriteRip.setPosition(x, 750);
        spritePlayer.setPosition(2300, 708); // hide player
    }

    // Update message text centering
    FloatRect details = messageText.getLocalBounds();
    float originX = details.left + (details.width / 2.0f);
    float originY = details.top + (details.height / 2.0f);
    messageText.setOrigin(originX, originY);

    //////////////////////////////////////

```

```
// RENDER SECTION

window.clear(); // clear screen

// Draw all background and foreground elements
window.draw(spriteBackground);
window.draw(spriteCloud);
window.draw(spriteCloud2);
window.draw(spriteCloud3);
window.draw(spriteTree);

for (int i = 0; i < numBranches; i++)
    window.draw(spriteBranch[i]);

window.draw(spritePlayer);
window.draw(spriteLog);
window.draw(spriteAxe);
window.draw(spriteRip);
window.draw(spriteBee);
window.draw(scoreText);
window.draw(messageText);
window.draw(timebar);

window.display(); // update display
}

return 0;
}
```

Pong Game without Class

Game Objectives

1. Prevent the Ball from Falling

- Control the **bat/paddle** at the bottom using the **Left** and **Right arrow keys**.
- Ensure the **ball bounces back** by hitting it with the bat.
- If the ball hits the **bottom wall** (misses the bat), the player **loses one life**.

2. Score Points

- When the ball hits the **top wall**, the player **earns 1 point**.
- The objective is to **score as many points as possible** before losing all lives.

3. Survive with Limited Lives

- The player starts with **3 lives**.
- Each time the ball hits the **bottom** (missed by the bat), **1 life is lost**.
- When all lives are lost:
 - The game shows "**Game Over .. Press Enter to restart**".
 - The player can press **Enter** to restart the game.

4. Game Controls

Action	Key
Move Bat Left	Left Arrow
Move Bat Right	Right Arrow
Restart After Game Over	Enter
Quit Game	Escape or Close window

After game over, pressing **Enter** resets:

- Score to 0
- Lives to 3
- Ball starts from the **top** with random X direction.

Game Code

```
#include <SFML/Graphics.hpp>
#include <iostream>
#include <sstream>
```



```

using namespace sf;
using namespace std;

int main()
{
    VideoMode video(800, 600);
    RenderWindow window(video, "Pong Game");

    float windowHeight = 1980;
    float windowHeight = 1020;
    View view(FloatRect(0, 0, windowHeight, windowHeight));

    window.setView(view);

    RectangleShape batShape;
    float batWidth = 200;
    float batHeight = 10;
    float batSpeed = 2;
    float batPixSec = windowHeight / batSpeed;
    batShape.setOrigin(batWidth / 2, batHeight / 2);
    batShape.setSize(Vector2f(batWidth, batHeight));
    batShape.setFillColor(Color::White);
    batShape.setPosition(windowWidth / 2, windowHeight - 20);

    CircleShape ballShape;
    float radius = 20;
    float ballSpeed = 3;
    float ballPixSecX = windowHeight / ballSpeed;
    float ballPixSecY = windowHeight / ballSpeed;
    ballShape.setRadius(radius);
    ballShape.setFillColor(Color::Magenta);
    ballShape.setPosition(20 + rand()%(int)(windowWidth-20) , 20);

    Font font;
    font.loadFromFile("KOMIKAP_.ttf");

    Text scoreText;
    scoreText.setFont(font);
    scoreText.setCharacterSize(75);
    scoreText.setFillColor(Color::White);
    scoreText.setPosition(20, 20);

    Text liveText;
    liveText.setFont(font);
    liveText.setCharacterSize(75);
    liveText.setFillColor(Color::White);

    float scoreVal = 0;
    float liveVal = 3;

```

```

int ballMoveX = rand()%2 == 0 ? -1 : 1;
int ballMoveY = 1;

bool paused = false;
bool bounceTop = false;
bool bounceBottom = true;
bool gameOver = false;

Clock ct;
Time dt;

while (window.isOpen())
{
    dt = ct.restart();
    Event ev;
    while (window.pollEvent(ev))
    {
        if (ev.type == Event::Closed || (ev.type == Event::KeyPressed
&& ev.key.code == Keyboard::Escape))
            window.close();
        if (ev.type == Event::KeyPressed && ev.key.code ==
Keyboard::Enter && gameOver)
        {
            paused = false;
            gameOver = false;
            bounceTop = false;
            bounceBottom = true;
            scoreVal = 0;
            liveVal = 3;
            ballShape.setPosition(windowWidth / 2, 20);
            ballMoveX = rand()%2 == 0 ? -1 : 1;
            ballMoveY = 1;
        }
    }

    stringstream ss, ss1;
    ss << "Score : " << scoreVal;
    scoreText.setString(ss.str());

    if(!gameOver)
        ss1 << "Lives : " << liveVal;
    else
        ss1 << "Game Over .. Press Enter to restart";
    liveText.setString(ss1.str());

    FloatRect details = liveText.getLocalBounds();
    liveText.setPosition(1980 - details.width - 20, 20);

    if (!paused)
    {

```

```

    if (Keyboard::isKeyPressed(Keyboard::Left))
    {
        float x = batShape.getPosition().x;
        float y = batShape.getPosition().y;
        x = x - dt.asSeconds() * batPixSec;
        if (x < batWidth / 2)
            x = batWidth / 2;
        batShape.setPosition(x, y);
    }
    if (Keyboard::isKeyPressed(Keyboard::Right))
    {
        float x = batShape.getPosition().x;
        float y = batShape.getPosition().y;
        x = x + dt.asSeconds() * batPixSec;
        if (x > windowWidth - batWidth / 2)
            x = windowWidth - batWidth / 2;
        batShape.setPosition(x, y);
    }
    float x = ballShape.getPosition().x;
    float y = ballShape.getPosition().y;
    x = x + dt.asSeconds() * ballPixSecX * ballMoveX;
    y = y + dt.asSeconds() * ballPixSecY * ballMoveY;
    if (x > windowWidth - radius)
        ballMoveX = -1;
    if (x < radius)
        ballMoveX = 1;
    if ((y > windowHeight - radius ||
batShape.getGlobalBounds().intersects(ballShape.getGlobalBounds())) &&
bounceBottom)
    {
        if (y > windowHeight - radius)
        {
            liveVal = liveVal - 1;
        }
        ballMoveY = -1;
        bounceTop = true;
        bounceBottom = false;
    }
    if (y < radius && bounceTop)
    {
        ballMoveY = 1;
        scoreVal = scoreVal + 1;
        bounceTop = false;
        bounceBottom = true;
    }
    ballShape.setPosition(x, y);
}
if(liveVal == 0)
{
    paused = true;

```

```
        gameOver = true;
        ballShape.setPosition(windowWidth/2, windowHeight/2);
    }

    window.clear();

    window.draw(batShape);
    window.draw(ballShape);

    window.draw(scoreText);
    window.draw(liveText);

    window.display();
}
}
```

Pong Game with Class

Ball.h File

```
#pragma once
#include<SFML/Graphics.hpp>
using namespace sf;

class Ball{
private:
    CircleShape ballShape;
    float pos_x, pos_y;
    float ballSpeed;
    float ballPixelPerSecX ;
    float ballPixelPerSecy;
    int moveX;//1: Right -1: left;
    int moveY;//1:Down -1:Up
public:
    Ball();
    CircleShape returnShape();
    void upadteBall(Time &dt);
    void ballBounceRight();
    void ballBounceLeft();
    void ballBounceTop(int &score, bool &incScore);
    void ballBounceBottom(bool &incScore, int &live);
    void ballTochesBat(FloatRect batBound, bool &incScore);
};
```

Ball.cpp File

```
#include "ball.h"
Ball::Ball()
{
    ballShape.setFillColor(Color::White);
    ballShape.setRadius(40);
    pos_x = 60 + rand()%1870;
    pos_y = 60;
    ballShape.setPosition(pos_x, pos_y);
    moveY= 1;
    moveX = 1;
    ballSpeed = 5;
    ballPixelPerSecX = 1980/ballSpeed;
    ballPixelPerSecy = 1020/ballSpeed;
}
CircleShape Ball::returnShape()
{
}
```

```

        return(ballShape);
    }

    void Ball::upadteBall(Time &dt)
    {
        float x = ballShape.getPosition().x;
        float y = ballShape.getPosition().y;
        x = x + moveX * dt.asSeconds()* ballPixelPerSecX;
        y = y + moveY * dt.asSeconds() * ballPixelPerSecy;
        ballShape.setPosition(x, y);
    }

    void Ball::ballBounceRight()
    {
        if(ballShape.getPosition().x > 1900)
            moveX = -1;
    }

    void Ball::ballBounceLeft()
    {
        if(ballShape.getPosition().x < 0)
            moveX = 1;
    }

    void Ball::ballBounceTop(int &score, bool &incScore)
    {
        if(ballShape.getPosition().y < 0 && incScore)
        {
            moveY = 1;
            score = score+1;
            incScore = false;
        }
    }

    void Ball::ballBounceBottom(bool &incScore, int &live)
    {
        if(ballShape.getPosition().y > 940 && !incScore)
        {
            moveY = -1;
            incScore = true;
            live = live -1;
        }
    }

    void Ball::ballTochesBat(FloatRect batBound, bool &incScore)
    {
        if(ballShape.getGlobalBounds().intersects(batBound) && !incScore)
        {
            moveY = -1;
            incScore = true;
        }
    }
}

```

Bat.h File

```
#pragma once
#include<SFML/Graphics.hpp>
using namespace sf;
class Bat{
private:
    RectangleShape batShape;
    float batSpeed;
    float batPixelPerSec;
public:
    Bat();
    void moveLeft(Time &dt);
    void moveRight(Time &dt);
    RectangleShape returnShape();
};
```

Bat.cpp File

```
#include "bat.h"
Bat::Bat()
{
    batShape.setFillColor(Color::White);
    batShape.setSize(Vector2f(400, 10));
    batShape.setPosition(1980 / 2.0f - 200, 1020 - 40);
    batSpeed = 5;
    batPixelPerSec = 1980 / batSpeed;
}

void Bat::moveLeft(Time &dt)
{
    float x = batShape.getPosition().x;
    float y = batShape.getPosition().y;
    x = x - (batPixelPerSec * dt.asSeconds());
    if (x < 0)
    {
        x = 0;
    }
    batShape.setPosition(x, y);
}

void Bat::moveRight(Time &dt)
{
    float x = batShape.getPosition().x;
    float y = batShape.getPosition().y;
    x = x + (batPixelPerSec * dt.asSeconds());
    if (x > 1580)
    {

```

```

        x = 1580;
    }
    batShape.setPosition(x, y);
}

RectangleShape Bat::returnShape()
{
    return(batShape);
}

```

PongGame.cpp File

```

#include <SFML/Graphics.hpp>
#include <sstream>
#include "bat.h"
#include "bat.cpp"
#include "ball.h"
#include "ball.cpp"

using namespace std;
using namespace sf;

int main()
{
    VideoMode vm(800, 600);
    RenderWindow window(vm, "Pong Game!!!");

    View vw(FloatRect(0, 0, 1980, 1020));
    window.setView(vw);

    bool paused = false;
    bool gameOver = false;
    bool acceptInput = true;

    int lives = 3;
    int score = 0;

    Text livesText, scoreText, messageText;
    Font ft;
    ft.loadFromFile("KOMIKAP_.ttf");
    scoreText.setFont(ft);
    scoreText.setFillColor(Color::White);
    scoreText.setCharacterSize(75);
    scoreText.setPosition(20, 20);
    scoreText.setString("Score = 0");

    livesText.setFont(ft);
    livesText.setCharacterSize(75);

```



```

livesText.setFill(Color::White);
livesText.setString("Lives = 3");

Bat bat;
Ball ball;

bool incScore = false;

Clock ct;
Time dt;

while (window.isOpen())
{
    dt = ct.restart();
    // game Logic
    Event ev;
    while (window.pollEvent(ev))
    {
        if (ev.type == Event::KeyReleased && !acceptInput)
        {
            acceptInput = true;
        }
        if (ev.type == Event::Closed)
        {
            window.close();
        }
    }
    if (acceptInput)
    {
        if (Keyboard::isKeyPressed(Keyboard::Space))
        {
            paused = !paused;
            acceptInput = false;
        }
        if (Keyboard::isKeyPressed(Keyboard::Enter))
        {
            gameOver = false;
            paused = false;
            // re-init
            lives = 3;
            score = 0;
            acceptInput = false;
            incScore = false;
            ball = Ball();
        }
    }
    if (!paused)
    {
        if (lives == 0)
        {

```

```

        gameOver = true;
        paused = true;
    }
    if (Keyboard::isKeyPressed(Keyboard::Left))
    {
        bat.moveLeft(dt);
    }
    if (Keyboard::isKeyPressed(Keyboard::Right))
    {
        bat.moveRight(dt);
    }
    stringstream str_score, str_lives;
    str_score << "Score = " << score;
    str_lives << "Lives = " << lives;
    scoreText.setString(str_score.str());
    livesText.setString(str_lives.str());

    FloatRect boundsLive = livesText.getLocalBounds();
    float y = 20;
    float x = vw.getSize().x - boundsLive.width - 20;
    livesText.setPosition(x, y);

    ball.upadteBall(dt);
    ball.ballBounceRight();
    ball.ballBounceLeft();
    ball.ballBounceTop(score, incScore);
    ball.ballBounceBottom(incScore, lives);
    ball.ballTochesBat(bat.returnShape().getGlobalBounds(),
incScore);
    }
    window.clear();
    // draw the sprites

    window.draw(bat.returnShape());
    window.draw(ball.returnShape());

    window.draw(scoreText);
    window.draw(livesText);
    window.display();
}
}

```